

EQUILOAN: FAIRNESS AWARE, CALIBRATED AND EXPLAINABLE LOAN APPROVAL DECISION SUPPORT SYSTEM

A PROJECT REPORT

Submitted by

KAMESHWAR M 210425243106

MITHRA VEERA A 210425243148

VIJAYARAGAVAN K 210425243316

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

CHENNAI INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Affiliated to Anna University, Chennai & Recognized by AICTE New Delhi)

Chennai-600 069

OCTOBER 2026



**CHENNAI
INSTITUTE OF TECHNOLOGY**
(Autonomous)



Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

- IM1.** To create next generation leader by effective teaching learning methodologies and instill scientific spark in them to meet the global challenges.
- IM2.** To transform lives through deployment of emerging technology, novelty and sustainability.
- IM3.** To inculcate human values and ethical principles to cater the societal needs.
- IM4.** To contribute towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R&D establishments.
- IM5.** To nurture incubation centres enabling structured entrepreneurship and start-ups.



**CHENNAI
INSTITUTE OF TECHNOLOGY**
(Autonomous)



Accredited by



Vision of the Department:

To Achieve excellent standards of quality-education by using the latest tools, nurturing collaborative culture and disseminating customer oriented innovations to relevant areas of academia and industry towards serving the greater cause of society.

Mission of the Department:

- IM1.** To develop professionals who are skilled in the area of Artificial Intelligence and Data Science.
- IM2.** To impart quality and value based education and contribute towards the innovation of computing, expert system, Data Science to raise satisfaction level of all stakeholders.
- IM3.** Our effort is to apply new advancements in high performance computing hardware and software.

CHENNAI INSTITUTE OF TECHNOLOGY (AUTONOMOUS)

(Affiliated to Anna University, Chennai & AICTE New Delhi)

Chennai-600 069

BONAFIDE CERTIFICATE

Certified that this project report “EQUILOAN: FAIRNESS AWARE, CALIBRATED AND EXPLAINABLE LOAN APPROVAL DECISION SUPPORT SYSTEM” is the bonafide work of KAMESHWAR M (210425243106), MITHRA VEERA A (210425243148) and VIJAYARAGAVAN K (210425243316), who carried out the project work under the supervision of Dr. G. Sanmugasundaram, Department of Artificial Intelligence and Data Science, Chennai Institute of Technology.

HEAD OF THE DEPARTMENT

Department of Artificial Intelligence and Data Science
Chennai Institute of Technology
Kundrathur, Chennai-600069

PROJECT SUPERVISOR

Dr. G. Sanmugasundaram
Department of Artificial Intelligence and Data Science
Chennai Institute of Technology

Certified that the above students have attended the viva voce during the examination held on

_____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our gratitude to the management of Chennai Institute of Technology for providing the facilities and opportunity to carry out this undergraduate project as part of the B.Tech Artificial Intelligence and Data Science programme. We thank the Principal, Dean and the Head of the Department for the academic support and laboratory facilities made available during the project period.

We sincerely thank our project supervisor, Dr. G. Sanmugasundaram, for the regular guidance, technical suggestions and reviews given during the development of this work. The discussions on data preparation, leakage, model evaluation, fairness, calibration and presentation helped us move from a simple loan prediction model to a more complete data science study.

We also thank the faculty members and laboratory staff of the Department of Artificial Intelligence and Data Science for their suggestions during reviews. We are grateful to our classmates for the discussions that helped us identify practical issues in the model and dashboard. Finally, we thank our families for their support and encouragement during the completion of the project.

TABLE OF CONTENTS

ABSTRACT	v
LIST OF FIGURES	vi
LIST OF TABLES	viii
SYMBOLS AND ABBREVIATIONS	ix
1. INTRODUCTION	1
1.1 Background	1
1.1.1 Loan approval and credit assessment	1
1.1.2 Data science perspective	2
1.2 Problem Statement	3
1.2.1 Our understanding of the problem	4
1.2.2 Planned approach	5
1.3 Loan Approval as a Data Science Problem	6
1.4 Objectives	7
1.5 Importance of Reliable Credit Decision Support	9
1.6 Role of AI and ML in Loan Approval	10
1.7 Significance	11
1.8 Scope	12
1.9 Limitations of Existing Systems	12
1.10 Overview of Proposed System	13
2. LITERATURE REVIEW	14
2.1 Existing Research	14

2.2 Comparative Analysis of Referred Papers	20
2.3 Key Observations	22
2.4 Gaps in Existing Knowledge	23
2.5 Novelty and Value Addition	24
3. METHODOLOGY	26
3.1 System Overview	26
3.2 Problem Solving Approach	27
3.3 Technologies Used	28
3.4 System Architecture	30
3.5 Data Collection and Description	32
3.6 Dataset Health Check	34
3.7 Target Leakage Audit	36
3.8 Data Preprocessing	38
3.9 Class Imbalance Strategy	40
3.10 Feature Engineering	42
3.11 Models and Experimental Setup	43
3.12 Model Training and Cross Validation	45
3.13 Probability Calibration	47
3.14 Fairness Analysis	48
3.15 Confidence Based Human Review	49
3.16 Explainability and Stability	50
3.17 EvoForge Version Analysis Layer	51
3.18 Existing System and Proposed Work	52
4. IMPLEMENTATION AND DEVELOPMENT PROCESS	54
4.1 Implementation Overview	54

4.2 Data Validation Module	55
4.3 Preprocessing Module	56
4.4 Baseline Modeling Module	57
4.5 Calibration Module	58
4.6 Fairness Module	59
4.7 Abstention Module	60
4.8 Robustness and Evaluation Module	61
4.9 Notebook Version History	62
4.10 Dashboard and Presentation Layer	64
4.11 Challenges and Mitigation	66
5. TESTING AND VALIDATION	68
5.1 Introduction to System Testing	68
5.2 Testing Objectives	69
5.3 Testing Strategy	70
5.4 Data Pipeline Validation	71
5.5 Unit Testing	72
5.6 Integration Testing	73
5.7 Machine Learning Validation	74
5.8 Leakage Validation	75
5.9 Imbalance Validation	76
5.10 Calibration and Fairness Validation	77
5.11 Abstention Validation	78
5.12 Reproducibility and Stability	79
5.13 Test Cases and Results	80
5.14 Testing Summary	81

6. RESULTS AND DISCUSSION	82
6.1 Dataset and Leakage Findings	82
6.2 Natural Distribution Results	84
6.3 Balanced Condition Results	86
6.4 Calibration Results	88
6.5 Fairness Results	89
6.6 Human Review Results	90
6.7 Feature Stability	91
6.8 Final Scorecard and Discussion	92
7. CONCLUSION	94
7.1 Summary of the Work	94
7.2 Key Contributions	95
7.3 Achievement of Objectives	96
7.4 Limitations	97
7.5 Future Enhancements	98
REFERENCES	100
APPENDIX	103

ABSTRACT

Loan approval prediction is a common classification problem in data science, but a high accuracy value alone does not show whether a model is reliable enough for decision support. This project develops EquiLoan as an undergraduate data science framework that studies the complete modeling process, starting from raw data validation and ending with model reliability checks. The dataset contains 148,670 records and 37 columns. The target variable is status, with 111,273 approved applications and 37,397 rejected applications. The original class distribution is approximately 74.85 percent approved and 25.15 percent rejected. A second experimental condition is created by random undersampling of the majority class to obtain 37,397 records from each class without changing any feature values. The implementation uses Python, pandas, NumPy, scikit-learn and XGBoost. Preprocessing is fitted inside the cross-validation folds so that imputation, encoding, scaling and outlier clipping do not use information from validation data. A dedicated leakage audit shows that `interest_rate_spread` and `upfront_charges` are missing for more than 95 percent of rejected applications but are mostly present for approved applications. Rate related variables are therefore removed from the main benchmark because they can reveal information generated after the underwriting decision. Logistic Regression, Decision Tree, Random Forest, XGBoost and Naive Bayes are compared using five-fold stratified cross-validation. On the natural distribution, XGBoost gives the strongest result among the tested models with mean accuracy 0.8682, F1 score 0.6757 and ROC-AUC 0.8571. On the balanced condition, its F1 score increases to 0.7589 and ROC-AUC is 0.8532, showing the trade-off between overall accuracy and minority-class learning. The project then studies probability calibration, demographic group differences, selective prediction and feature stability. Isotonic calibration gives the lowest Brier score on the natural experiment, while the uncalibrated model already has a low expected calibration error. Fairness screening on the natural distribution produces DIR values above 0.93 for gender, region and age, while the balanced experiment changes the group prediction pattern and gives an age DIR of 0.749. The abstention study shows that sending uncertain cases for human review can improve accuracy on the cases decided automatically, but this improvement comes with lower coverage. The novelty of the student project is therefore not a new mathematical algorithm. It is the controlled combination of leakage analysis, natural versus balanced class experiments, calibration, fairness screening, uncertainty handling and stability analysis in one reproducible workflow.

Keywords: Loan Approval, Data Science, Machine Learning, Target Leakage, Class Imbalance, Probability Calibration, Fairness, Explainable AI, Human Review, Cross Validation

LIST OF FIGURES

- Figure 1.1 Target Distribution in the Original Dataset
- Figure 3.1 Missing Values Before Pipeline Imputation
- Figure 3.2 EquiLoan Data Science Workflow
- Figure 3.3 Dual Experimental Condition Strategy
- Figure 3.4 Three State Decision Layer
- Figure 3.5 EvoForge Version Analysis Concept
- Figure 4.1 Modular Source Code Structure
- Figure 4.2 Early Notebook Output Showing Missing Data
- Figure 4.3 Early Model Comparison Before Final Leakage Audit
- Figure 6.1 Natural Distribution Baseline Model Comparison
- Figure 6.2 Balanced Condition Baseline Model Comparison
- Figure 6.3 Probability Calibration Comparison
- Figure 6.4 Fairness Screening Across Conditions
- Figure 6.5 Selective Prediction Coverage and Accuracy
- Figure 6.6 Stable Feature Importance Across Folds
- Figure 6.7 Final Multi-Criteria Scorecard

LIST OF TABLES

Table 1.1	Dataset Summary
Table 1.2	Main Data Science Risks
Table 2.1	Literature Comparison
Table 2.2	Novelty Matrix
Table 3.1	Feature Groups
Table 3.2	Missing Value Summary
Table 3.3	Target Leakage Audit
Table 3.4	Models Included in the Study
Table 3.5	Evaluation Metrics
Table 3.6	Existing System and Proposed Work
Table 4.1	Project Module Structure
Table 4.2	Notebook Version History
Table 5.1	Core Test Cases
Table 5.2	Validation Checklist
Table 6.1	Natural Distribution Results
Table 6.2	Balanced Condition Results
Table 6.3	Calibration Results
Table 6.4	Fairness Audit
Table 6.5	Abstention Results
Table 6.6	Feature Stability Summary
Table 6.7	Final Research Scorecard

SYMBOLS AND ABBREVIATIONS

Acronym	Expansion
AI	Artificial Intelligence
ML	Machine Learning
DS	Data Science
LR	Logistic Regression
DT	Decision Tree
RF	Random Forest
XGB	Extreme Gradient Boosting
NB	Naive Bayes
SVM	Support Vector Machine
CV	Cross Validation
SHAP	SHapley Additive exPlanations
SPD	Statistical Parity Difference
DIR	Disparate Impact Ratio
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
PR-AUC	Area Under the Precision Recall Curve
ECE	Expected Calibration Error
IQR	Interquartile Range
XAI	Explainable Artificial Intelligence

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Loan approval is an important activity in banking because every approved application creates both an opportunity and a risk. A bank earns interest when a borrower repays according to the agreed schedule, but an unsuitable approval can increase credit losses. At the same time, rejecting a suitable applicant can also create a lost business opportunity. For this reason, the loan approval process uses information about the applicant, the requested loan, income, credit history, property details and other financial factors.

Machine learning is useful in this setting because it can learn patterns from earlier applications and produce a prediction for a new record. However, a loan approval project is not complete when a classifier prints an accuracy value. A data scientist has to ask whether the input fields are available before the decision, whether the class distribution is handled correctly, whether the model performs consistently on different folds, whether its probability values are reliable, and whether group-level results show large differences. These questions form the main motivation for EquiLoan.

This project was initially started as a conventional loan prediction exercise. During later notebook versions, the work changed after we observed unusually strong scores and large missingness differences in rate-related columns. This led to a separate leakage audit and a cleaner experimental pipeline. The final project therefore records not only the selected model but also the mistakes and design changes that were found during development.

1.1.1 Loan Approval and Credit Assessment

Credit assessment traditionally combines documentary checks, credit records, repayment capacity and the terms of the requested loan. In a machine learning dataset, these ideas appear as numerical and categorical features. Some variables describe the applicant, while others describe the product or the property. The target status tells whether the historical application was approved or rejected. The model learns a statistical relationship between the input features and this historical decision. It does not learn a universal rule of who deserves credit, and it cannot replace institutional policy or legal review.

1.1.2 Data Science Perspective

From a data science perspective, the main challenge is to build a fair comparison between candidate models. The same folds, preprocessing rules and feature restrictions must be used so that one model is not accidentally given more information than another. Performance must also be described using more than one metric because the data is imbalanced. These requirements are reflected throughout the methodology and testing chapters.

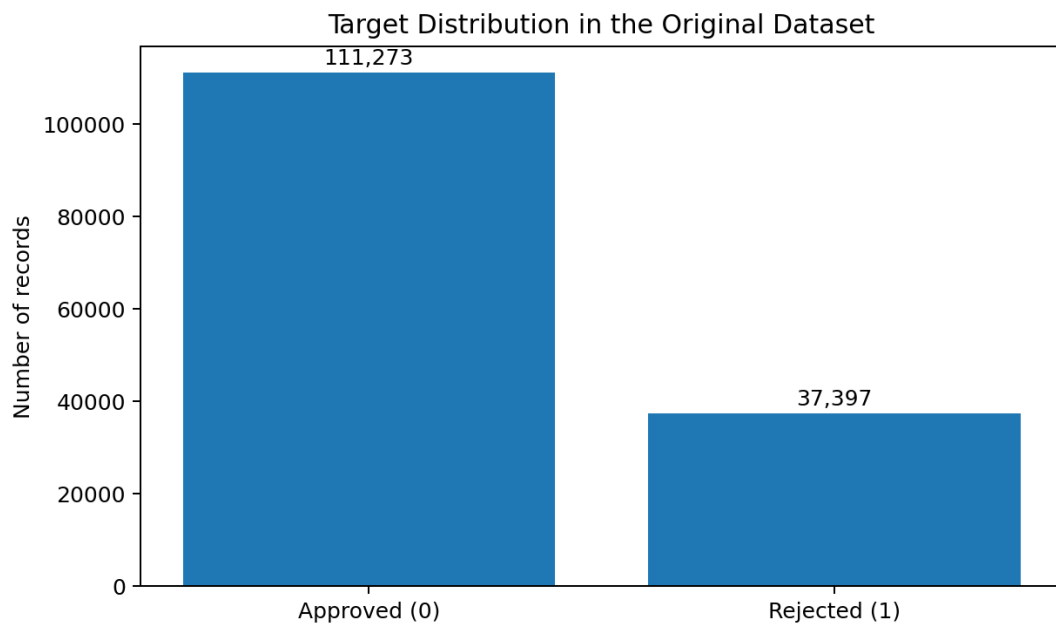


Figure 1.1 Target Distribution in the Original Dataset

1.2 PROBLEM STATEMENT

Many student loan prediction projects use a cleaned public dataset, compare a few classifiers and select the one with the highest accuracy. That procedure is useful for learning basic classification, but it can hide important issues. In the supplied dataset, the target is not balanced, several fields contain missing values, and some interest-related variables are closely connected with the stage at which the decision is made. If those fields are used without checking their timing, the model may appear to predict the target while actually reading information that becomes available after the decision.

The problem addressed in this project is therefore to predict loan status using a reproducible and leakage-aware data science workflow and to study the reliability of the prediction. The study compares the natural class distribution with a balanced condition, evaluates several standard classifiers, examines probability calibration and group-level fairness, and introduces a human review zone for uncertain cases. The goal is not to automate

a real bank. The goal is to understand how a student machine learning model changes when these practical checks are added.

1.2.1 OUR UNDERSTANDING OF THE PROBLEM

The first issue is data quality. Missing values are not distributed evenly across the columns, and a few variables have missing rates above twenty percent. A preprocessing decision can therefore change the model considerably. In EquiLoan, preprocessing is fitted inside the training folds so that the model does not use information from validation data.

The second issue is class imbalance. The original data contains 111,273 approved applications and 37,397 rejected applications. A model can obtain a high overall accuracy by learning the majority class more easily. We therefore report recall, F1 score and ROC-AUC in addition to accuracy. A balanced experimental dataset is used as a separate condition rather than as a replacement for the original distribution.

The third issue is target leakage. The leakage audit shows that `interest_rate_spread` and `upfront_charges` are missing for nearly all rejected records but are present for most approved records. This pattern is not a normal predictive signal from the application stage. It is likely connected with underwriting information that is produced only when a loan proceeds further. These variables are removed from the main benchmark.

The fourth issue is model confidence. A predicted probability of 0.80 is useful only if similar cases are actually correct at approximately that rate. Calibration analysis is therefore included after the baseline model comparison. Finally, a low-confidence prediction is not forced into an automatic decision in the abstention experiment. It can be marked for human review.

1.2.2 PLANNED APPROACH

The project begins with a raw CSV and performs a health check before any modeling step. Dataset dimensions, target distribution, missing values and duplicate rows are recorded. A dedicated leakage audit is then used to identify fields that should not be used as predictors.

The modeling stage uses a common preprocessing pipeline and five-fold stratified cross-validation. Logistic Regression, Decision Tree, Random Forest, XGBoost and Naive Bayes are compared under the natural and balanced class conditions. XGBoost is then studied further because it gives the strongest overall baseline in the completed experiments.

After baseline evaluation, the work moves to calibration, fairness screening, abstention and feature stability. These analyses answer different questions, so they are not collapsed into

one accuracy value. The final dashboard presents the results as a decision-support study rather than a commercial lending tool.

1.3 LOAN APPROVAL AS A DATA SCIENCE PROBLEM

The project uses the standard stages of a data science life cycle: understanding the data, preparing a reproducible transformation pipeline, building baseline models, evaluating generalization, interpreting the model and communicating the results. The most important difference from a small demonstration is that each stage is treated as an experiment. For example, class balancing is not assumed to be an improvement. It is measured against the natural distribution. Calibration is not assumed to improve every metric. It is tested using Brier score, log loss and expected calibration error. This experimental approach is the main academic value of the project.

1.4 OBJECTIVES

1.4.1 General Objective

To design and evaluate a reproducible loan approval prediction framework using data science methods, with specific attention to data leakage, class imbalance, model performance, probability calibration, fairness screening, uncertainty handling, interpretability and stability.

1.4.2 Specific Objectives

1. To inspect the raw loan dataset and document its structure, missing values, target distribution and possible leakage fields.
2. To implement training-fold-only preprocessing for numerical and categorical variables.
3. To compare standard machine learning classifiers under the same five-fold stratified cross-validation procedure.
4. To compare the original target distribution with a separate 50:50 experimental condition created without synthetic values.
5. To evaluate model quality using accuracy, precision, recall, F1 score, ROC-AUC and Brier score.
6. To compare uncalibrated probabilities with Platt scaling and isotonic regression.
7. To audit group-level prediction differences using gender, region and age where these attributes are available.
8. To test a three-state decision process in which uncertain cases are sent for human review.
9. To evaluate feature stability across folds and retain the version history of the project as evidence of model development.

1.5 IMPORTANCE OF RELIABLE CREDIT DECISION SUPPORT

Reliability is important because an incorrect prediction can have different consequences depending on the type of error. In the coding used in this project, status 1 represents rejection. Missing a rejected case and incorrectly predicting approval is different from incorrectly rejecting an approved case. A single accuracy value combines these mistakes and therefore cannot describe them fully. The project uses precision, recall and F1 to make class-specific behaviour visible.

Reliable decision support also requires honest probability values. Ranking metrics such as ROC-AUC measure whether higher-risk cases tend to receive higher scores, but they do not show whether a score of 0.70 actually behaves like a 70 percent probability. Calibration metrics answer this second question. By including both ranking and calibration, EquiLoan gives a more complete view of model quality.

1.6 ROLE OF AI AND ML IN LOAN APPROVAL

Machine learning can process a large tabular dataset and detect interactions that are difficult to describe with a fixed manual rule. Tree ensembles are especially useful for nonlinear relationships and mixed feature effects. Logistic Regression remains useful because it provides an interpretable linear baseline. Naive Bayes provides a simple probabilistic comparison. The purpose of testing several models is not to prove that one algorithm is universally best, but to measure how the same dataset behaves under different assumptions.

1.7 SIGNIFICANCE

The project is significant mainly as a student data science study. It connects topics that are often learned separately: missing-data handling, leakage prevention, imbalanced classification, cross-validation, probability calibration, fairness metrics, explainability and uncertainty. It also shows why very high reported accuracy should be checked carefully before it is accepted as evidence of a strong model.

The project is linked to SDG 8 and SDG 10 at a limited academic level. More consistent credit analysis can support efficient financial services, while group-level fairness checks can reveal whether model outputs vary strongly across available demographic categories. We do not claim that this prototype solves financial inclusion or social inequality. Those outcomes depend on policy, data collection and real lending practice beyond the scope of this project.

1.8 SCOPE

- The study is limited to supervised binary classification on the supplied tabular dataset.
- The model is an academic decision-support prototype and is not intended for production lending.
- The project compares standard machine learning models and does not introduce a new mathematical classifier.
- Fairness analysis is limited to the demographic or proxy fields present in the data.
- Counterfactual recommendations are discussed conceptually; protected attributes are not treated as changeable applicant actions.
- The balanced condition is created by undersampling the majority class and is used only as a controlled experiment.

1.9 LIMITATIONS OF EXISTING SYSTEMS

The referred literature shows strong progress in loan prediction, feature selection and explainability, but many studies still focus mainly on model accuracy. Some papers report extremely high accuracy values on public loan datasets. Such values are not automatically wrong, but they create a need to examine whether duplicate records, target leakage or post-decision attributes are present. Recent studies also show that SHAP, LIME, ensemble learning and web deployment are already common research directions. For that reason, these components are not treated as novelty by themselves in EquiLoan.

1.10 OVERVIEW OF THE PROPOSED SYSTEM

EquiLoan begins with raw data validation. After leakage checking, training-fold-only preprocessing is applied. Baseline models are evaluated under natural and balanced conditions. XGBoost is taken forward for calibration, fairness screening, abstention and stability analysis because it provides the strongest overall baseline among the models tested in the final codebase. The system then produces tables and plots for the report and dashboard. The final output is a research summary that explains performance, reliability and limitations rather than a simple approve-or-reject answer.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the research papers referred to during the project. The papers were selected because they cover the main directions relevant to loan approval prediction: basic classification, ensemble learning, feature selection, cross-validation, explainability and recent hybrid approaches. The review is used to position EquiLoan as a student data science project and to avoid claiming that established methods are new.

2.1.1 Bank Loan Prediction Using Machine Learning Techniques

F. M. Ahosanul Haque and Md. Mahedi Hassan use a dataset of 148,670 instances and 37 attributes, which matches the dimensions of the dataset used in our project. They compare Decision Tree, AdaBoost, Random Forest, SVM and Gaussian Naive Bayes and report 99.99 percent accuracy for AdaBoost. This paper is important because it provides a direct baseline on the same dataset family. It also motivates our leakage audit. A score close to 100 percent is impressive, but it should be checked against the timing of features and missingness patterns before being used as evidence that the problem is solved.

2.1.2 A Prediction of Modernized Loan Approval System Based on Machine Learning Approach

M. Venkata Naga Lakshmi and P. Srinivasa Rao present a loan approval system based on historical applicant information from Kaggle. Their work discusses Naive Bayes, Decision Tree and MLP within an ensemble-oriented approach and targets approximately 90 percent accuracy. The study represents a common loan prediction design in which the main objective is to automate the decision and reduce manual effort. For EquiLoan, the paper is useful as an example of an accuracy-centred system that can be extended with calibration and reliability checks.

2.1.3 A Comparative Study of Loan Approval Prediction Using Machine Learning Methods

Vahid Sinap compares machine learning methods and studies feature selection using K-Best and Recursive Feature Elimination. The study reports the strongest accuracy for Random Forest and notes that RFE improves model performance. It also reports that cross-validation provides more consistent performance measurement than a single training, testing

and validation split. This finding directly supports our decision to use five-fold stratified cross-validation and to treat feature stability as a separate part of model evaluation.

2.1.4 A Data Driven Model for Predicting Loan Approval Using Machine Learning Approaches

Shady Abdelhady and co-authors use a dataset of 5,000 applicants with 14 attributes and compare Decision Tree, KNN and SVM. They report SVM accuracy of 96 percent, Decision Tree accuracy of 92 percent and KNN accuracy of 86 percent. The work shows that different algorithms can perform very differently even on the same task. It also demonstrates the importance of evaluating more than one baseline instead of selecting an algorithm in advance.

2.1.5 Prediction of Loan Approval in Banks using Machine Learning Approach

Viswanatha, Ramachandra, Vishwas and Adithya compare Random Forest, Naive Bayes, Decision Tree and KNN and report Naive Bayes as the strongest method in their experiment with an accuracy of 83.73 percent. This is useful for our study because it shows that simple probabilistic models can remain competitive depending on the dataset. We therefore retain Naive Bayes as a baseline rather than testing only ensemble methods.

2.1.6 Loan Approval Prediction Using Machine Learning

Goliya Bhavani describes loan approval as a classification problem using variables such as credit score, income, age, marital status and gender. The paper discusses Logistic Regression, Random Forest and Naive Bayes as practical methods for reducing manual effort. It is representative of small student-level loan prediction projects and helps define the baseline that EquiLoan is intended to extend.

2.1.7 Performance Analysis of Ensemble Learning and Feature Selection Methods in Loan Approval Prediction at Banks

Muhammad Iqbal and co-authors evaluate several ensemble methods on a Kaggle loan dataset, including XGBoost, LightGBM, Gradient Boosting, Random Forest, AdaBoost and Extra Trees. They report very high XGBoost performance, including accuracy of 0.9974 and AUC of 0.9998. The paper confirms that ensemble models are strong baselines, but it also shows that high benchmark scores have already been reported. Therefore, simply using XGBoost cannot be presented as novelty.

2.1.8 Advancing Loan Approval Prediction With SHAP-Guided Feature Selection and LIME-Based Model Interpretability

Raisa Akter and co-authors combine SHAP-guided feature selection with LIME-based interpretation in a multiclassifier framework and deploy the results through a web application. They report logistic regression accuracy of 86.17 percent with 81 percent AUC on one dataset and 99.06 percent accuracy on another. This 2026 work is particularly important for our novelty audit because it shows that SHAP, LIME, model comparison and web deployment are already established together in recent loan approval research.

2.1.9 An Explainable Hybrid Machine Learning Framework for Loan Approval Prediction

Oktay Can Sevimgin and Denizhan Demirkol present a 2026 framework that evaluates nine classifiers, uses Grey Wolf Optimizer for hyperparameter selection and combines Random Forest with LightGBM. Their final hybrid model achieves 92.17 percent accuracy on an independent holdout test set, and SHAP is used for explainability. This paper shows that hybrid ensemble design, optimization and explainability are already active research directions. EquiLoan therefore focuses its value addition on experimental reliability checks rather than claiming a new hybrid algorithm.

2.2 COMPARATIVE ANALYSIS OF REFERRED PAPERS

Table 2.1 Literature Comparison

Study	Dataset	Models	Main focus	Relevance to EquiLoan
Haque & Hassan	148,670 x 37	DT, AdaBoost, RF, SVM, GNB	Accuracy focus	Same dataset family; high score motivates leakage audit
Lakshmi & Rao	Kaggle	NB, DT, MLP	Ensemble / accuracy	Automation baseline
Sinap	Loan dataset	RF, SVM, DT	RFE, K-Best, CV	Supports feature selection and cross-validation
Abdelhady et al.	5,000 x 14	DT, KNN, SVM	Accuracy, precision, recall, F1	Shows model-dependent performance
Viswanatha et al.	Public loan data	RF, NB, DT, KNN	Accuracy comparison	Keeps simple baselines relevant
Bhavani	Public loan data	LR, RF, NB	Basic classification	Student-project baseline
Iqbal et al.	4,173 usable	XGB, LGBM, GB, RF, AdaBoost, ET	Ensemble + feature selection	XGBoost already strong in literature
Akter et al.	Two Kaggle datasets	Multiple classifiers	SHAP, LIME, web app	XAI and deployment already established
Sevimgin & Demirkol	Kaggle	9 models + RF-LGBM hybrid	GWO + SHAP + nested CV	Hybrid optimization already established

2.3 KEY OBSERVATIONS FROM EXISTING RESEARCH

- Ensemble tree methods are consistently strong baselines, but the best algorithm changes with dataset and preprocessing.
- Feature selection can improve performance, and cross-validation is more reliable than a single random split.
- Very high accuracy values are already reported in loan datasets. High accuracy alone is therefore not a useful novelty claim.
- Explainability through SHAP and LIME is now common in recent work, including web-based applications.
- Hybrid models and optimization methods such as Grey Wolf Optimizer are also already present in 2026 literature.
- Calibration, abstention and cross-fold stability receive much less attention in the specific papers referred to for this project.

2.4 GAPS IN EXISTING KNOWLEDGE

The literature does not support a claim that Random Forest, XGBoost, SHAP, LIME, web dashboards or hybrid ensemble learning are new. These methods have already been studied in loan approval prediction. The gap used in this student project is narrower and more practical. EquiLoan studies several reliability questions together on the same large dataset and records the trade-offs instead of choosing a model only from accuracy.

The first gap is the handling of target leakage. The same 148,670 by 37 dataset family can produce extremely high accuracy when post-underwriting fields are included. EquiLoan makes the leakage audit an explicit experimental stage and removes the identified rate-related variables from the main benchmark. The second gap is the controlled comparison between the natural target distribution and a separate balanced condition. The third gap is the combination of probability calibration, demographic screening, confidence-based abstention and feature stability after baseline model selection.

2.5 NOVELTY AND VALUE ADDITION

The novelty of EquiLoan is experimental and methodological rather than algorithmic. The project does not claim a new classifier. Its value comes from placing several reliability checks into one reproducible student workflow and then showing how the conclusions change when the data distribution or decision rule changes. This is a more defensible contribution than calling a standard ensemble model novel.

Table 2.2 Novelty Matrix

Area	Common approach	EquiLoan value addition
Prediction	Choose highest accuracy model	Compare several metrics and inspect class-specific behaviour
Leakage	Often not discussed	Dedicated audit of post-underwriting fields
Class balance	Balance once and train	Compare natural and 50:50 conditions separately
Probability	Use raw classifier probability	Compare uncalibrated, Platt and isotonic calibration
Fairness	Optional or absent	Group-level SPD and DIR screening under both conditions
Decision rule	Binary approve/reject	Add an uncertain human-review band
Robustness	Single split	Five-fold CV plus feature stability analysis
Version history	Final notebook only	Keep multiple notebook iterations and trace model changes

CHAPTER 3

METHODOLOGY

3.1 SYSTEM OVERVIEW

The methodology follows a standard data science life cycle, but each important modeling choice is treated as a controlled experiment. The system starts with the untouched loan dataset, checks its structure and missingness, audits possible leakage and then builds preprocessing pipelines that are fitted only on training folds. Baseline models are evaluated on the natural distribution and on a separate balanced condition. Reliability analysis is performed only after the baseline comparison is complete.

Figure 3.2 EquiLoan Data Science Workflow (editable representation)

Stage	Output
Raw data	148,670 x 37 reference dataset
Validation	health report and leakage audit
Preprocessing	training-fold-only transformations
Baseline models	five-fold comparison
Reliability analysis	calibration, fairness, abstention, stability
Presentation	tables, plots and dashboard

3.2 PROBLEM SOLVING APPROACH

The work is divided into two main experimental conditions. Condition A keeps the original class distribution. Condition B contains the same number of approved and rejected rows, created by random undersampling of the majority class using a fixed seed. No synthetic applicant values are generated for the balanced file. The two conditions are evaluated separately because balancing can change both predictive performance and group-level statistics.

Figure 3.3 Dual Experimental Condition Strategy

Condition	Rows	Class 0	Class 1	Purpose
Natural	148,670	111,273	37,397	Real reference distribution
Balanced	74,794	37,397	37,397	Controlled class-balance experiment

3.3 TECHNOLOGIES USED

3.3.1 Python

Python is used as the main programming language for data loading, preprocessing, model training, evaluation and report preparation. The language was selected because the project is a tabular data science problem and the required libraries are well supported.

3.3.2 Pandas and NumPy

Pandas is used for dataset inspection, target distribution analysis, missing-value summaries and result tables. NumPy supports numerical operations and the custom outlier clipping transformer.

3.3.3 Scikit-learn

Scikit-learn provides Pipeline, ColumnTransformer, imputation, one-hot encoding, scaling, cross-validation and evaluation functions. Keeping these operations inside a pipeline is central to the leakage-safe design.

3.3.4 XGBoost

XGBoost is included as a high-performance tree ensemble baseline. It is not assumed to be the best in advance. It is taken forward only after the five-fold comparison shows the strongest overall baseline among the tested models.

3.3.5 Matplotlib

Matplotlib is used to create the result figures included in the report. Plots are generated from saved result CSV files so that the figures can be reproduced from the experiment outputs.

3.3.6 Jupyter Notebook and Modular Python Files

The project retains multiple notebook versions because they show the development history. The final repeatable experiments are implemented in separate Python modules for validation, preprocessing, modeling, calibration, fairness, abstention, robustness and final evaluation.

3.4 SYSTEM ARCHITECTURE

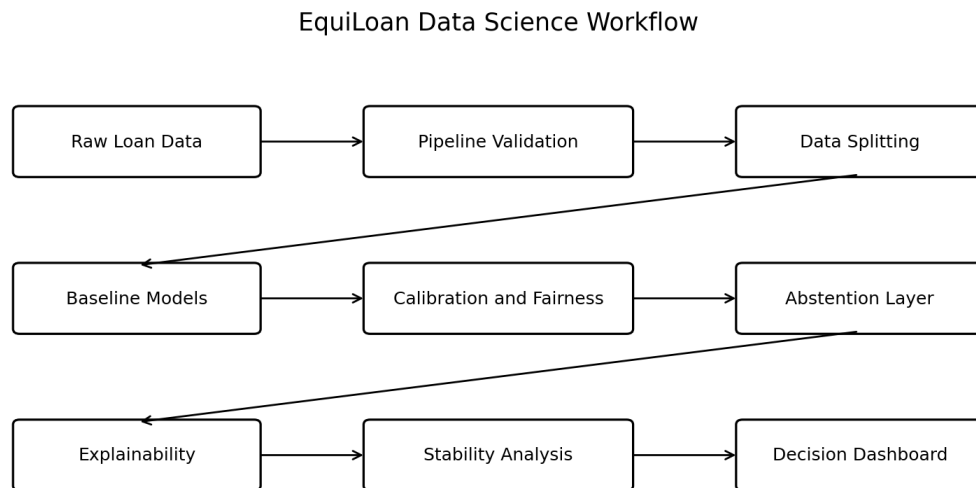


Figure 3.2 EquiLoan System Architecture

The architecture separates data preparation from model evaluation and separates predictive performance from reliability analysis. This separation is intentional. A model cannot be considered reliable merely because it has good predictive metrics, and fairness or calibration should not be measured before the pipeline itself is checked for leakage.

3.5 DATA COLLECTION AND DESCRIPTION

The project uses the supplied loan dataset stored as `loan_v2 (real distribution).csv`. It contains 148,670 rows and 37 columns. The dataset mixes numeric and categorical information related to loan terms, property values, applicant information, credit information and the final status. The file is kept as an immutable reference and is not manually cleaned before modeling.

Table 3.1 Feature Groups

Feature group	Examples
Identifiers	Unnamed: 0, id, year
Applicant	gender, age, region, senior_age
Loan details	loan_limit, loan_type, loan_purpose, loan_amount, term
Financial	income, property_value, ltv, dtir1
Credit	credit_score, credit_type, credit_worthiness, co-applicant_credit_type

Feature group	Examples
Other terms	neg_ammortization, interest_only, lump_sum_payment, occupancy_type, secured_by, total_units
Potential leakage	rate_of_interest, interest_rate_spread, upfront_charges, high_interest_rate
Target	status

3.6 DATASET HEALTH CHECK

The health check confirms that the source file has 148,670 rows, 37 columns and no duplicate rows. The target has two labels. Missing values are concentrated in a limited set of fields rather than being spread uniformly across the dataset. Upfront charges, interest rate spread and debt-to-income ratio contain the largest missing proportions.

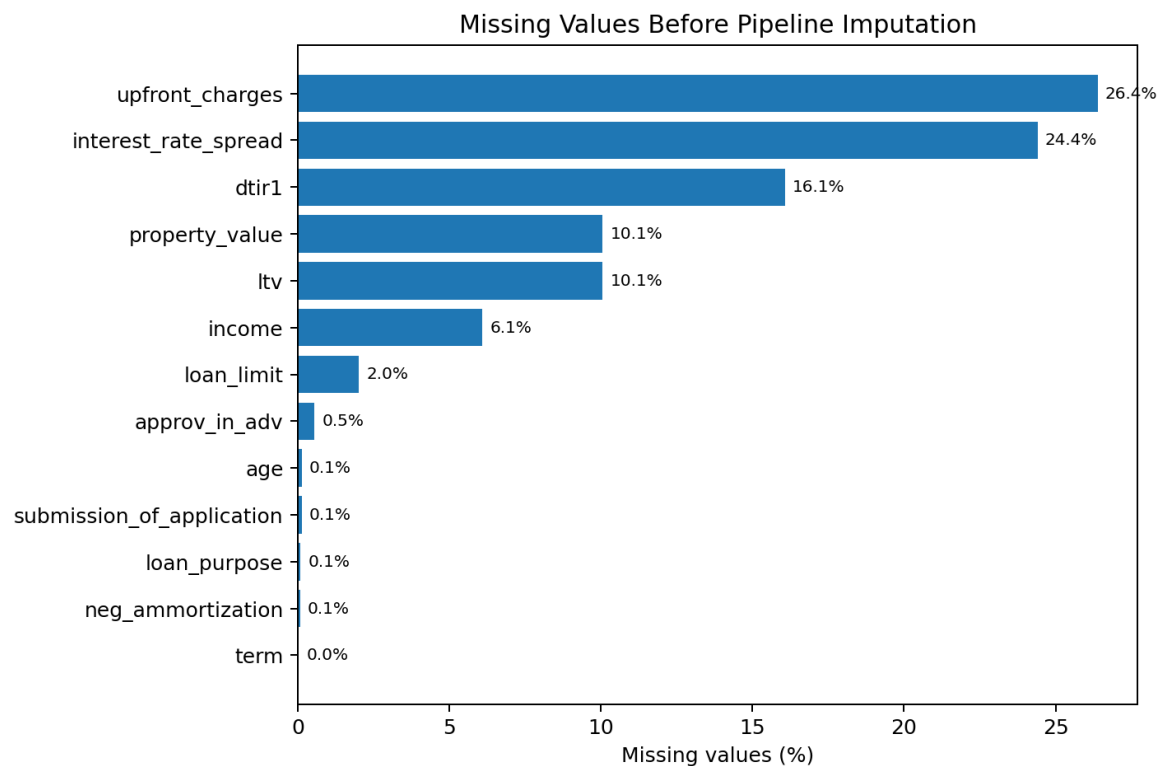


Figure 3.1 Missing Values Before Pipeline Imputation

Table 3.2 Missing Value Summary

Feature	Missing count	Missing percent
upfront_charges	39,231	26.39%
interest_rate_spread	36,283	24.41%
dtir1	23,899	16.08%
property_value	14,952	10.06%

Feature	Missing count	Missing percent
ltv	14,942	10.05%
income	9,046	6.08%
loan_limit	2,997	2.02%
approv_in_adv	815	0.55%
age	181	0.12%
submission_of_application	180	0.12%

3.7 TARGET LEAKAGE AUDIT

Target leakage occurs when a predictor contains information that would not be available at the time the prediction is supposed to be made. In loan data, a field created after underwriting can reveal whether the application progressed. The EquiLoan validator therefore checks the missingness of rate-related columns separately for approved and rejected records and also reports their correlation with status.

Table 3.3 Target Leakage Audit

Feature	Missing in approved	Missing in rejected	Correlation	Action
rate_of_interest	0.00%	0.00%	-0.5818	Removed from main benchmark
interest_rate_spread	0.32%	96.06%	0.2081	Removed from main benchmark
upfront_charges	3.10%	95.67%	0.2233	Removed from main benchmark
high_interest_rate	0.00%	0.00%	-0.3002	Removed from main benchmark

The missingness pattern is especially strong for `interest_rate_spread` and `upfront_charges`. More than 95 percent of rejected records are missing these values, while approved records are rarely missing them. A classifier can exploit this pattern without learning the applicant information that should support a decision. The final preprocessing module therefore excludes these four rate-related variables along with identifier columns. This decision also explains why our cleaned baseline scores are lower than some early notebook outputs and some published results on the same dataset family.

3.8 DATA PREPROCESSING

Preprocessing is implemented with a scikit-learn ColumnTransformer. Numerical variables use median imputation. The project also includes an IQR outlier clipper that learns its lower and upper fences from the training fold only. Standard scaling is applied after imputation and clipping. Categorical variables use most-frequent imputation followed by one-hot encoding with unknown categories ignored.

The most important rule is that preprocessing is fitted separately inside each cross-validation fold. The validation fold is transformed using parameters learned from the corresponding training fold. This prevents information from validation data from influencing the imputation median, category encoding, scaling or outlier fences.

3.9 CLASS IMBALANCE STRATEGY

The natural data contains approximately three approved records for each rejected record. We considered several possible approaches, including class weights, undersampling and synthetic oversampling. For the completed report, the direct comparison uses the natural dataset and an undersampled 50:50 condition. This choice allows us to study the effect of equal class representation without generating artificial applicant values.

The balanced condition is not treated as automatically better. It reduces the number of majority-class records and therefore changes the information available to the model. The final discussion compares recall, F1, ROC-AUC, calibration and fairness results under both conditions rather than choosing a dataset based only on accuracy.

3.10 FEATURE ENGINEERING

Feature engineering is kept conservative because the dataset already includes loan-to-value and debt-to-income style variables. Creating duplicate ratios can increase collinearity without adding information. New features are considered only when they have a clear meaning and can be calculated before the lending decision. In the final reproducible benchmark, the main focus is on the original pre-decision variables after leakage removal.

3.11 MODELS AND EXPERIMENTAL SETUP

Table 3.4 Models Included in the Study

Model	Reason for inclusion
Logistic Regression	Linear and interpretable baseline
Decision Tree	Nonlinear baseline with readable decision structure

Model	Reason for inclusion
Random Forest	Bagging ensemble for nonlinear tabular data
XGBoost	Boosted-tree baseline with strong tabular performance
Naive Bayes	Simple probabilistic baseline

The Decision Tree is limited to a maximum depth of 10 in the baseline code. Random Forest uses 100 trees. XGBoost uses 100 estimators with log-loss evaluation. Logistic Regression uses a maximum of 1000 iterations. These settings are intentionally moderate because the objective is to compare the complete pipeline rather than to perform an expensive hyperparameter search.

3.12 MODEL TRAINING AND CROSS VALIDATION

Five-fold StratifiedKFold cross-validation is used with shuffling and a fixed random state of 42. Stratification keeps the target proportions similar across folds. For each fold, the preprocessing pipeline is fitted on the training portion, the classifier is fitted on the transformed training data, and metrics are calculated on the transformed validation data. Mean and standard deviation across folds are stored in CSV result files.

Table 3.5 Evaluation Metrics

Metric	Purpose
Accuracy	Overall proportion of correct predictions
Precision	Correct positive predictions among predicted positives
Recall	Detected positives among actual positives
F1 score	Harmonic mean of precision and recall
ROC-AUC	Ranking ability across thresholds
Brier score	Mean squared error of predicted probabilities
Log loss	Penalty for incorrect confident probabilities
ECE	Difference between confidence bins and observed frequency

3.13 PROBABILITY CALIBRATION

After model selection, the probability outputs of XGBoost are compared with Platt scaling and isotonic regression. Platt scaling learns a sigmoid mapping from the model score to probability. Isotonic regression learns a monotonic non-parametric mapping. Calibration

is evaluated separately because a classifier can rank cases well while still producing poorly scaled probabilities.

3.14 FAIRNESS ANALYSIS

Fairness analysis uses available group attributes as diagnostic variables. The completed audit compares gender, region and age. For each attribute, the code identifies the group with the highest positive prediction rate and the group with the lowest rate. Statistical parity difference is the difference between these rates, while disparate impact ratio is the lower rate divided by the higher rate. A value below 0.80 is used only as a screening alert in this project. It is not presented as proof of unlawful discrimination.

3.15 CONFIDENCE BASED HUMAN REVIEW

The abstention experiment adds a third outcome between the two automatic decisions. A record is decided automatically when its probability is outside a defined uncertainty band. Records inside the band are marked for human review. The experiment reports automated coverage, review percentage, accuracy on the decided cases, false approval rate and false rejection rate. Wider uncertainty bands send more records for review and generally improve accuracy on the remaining automated cases.

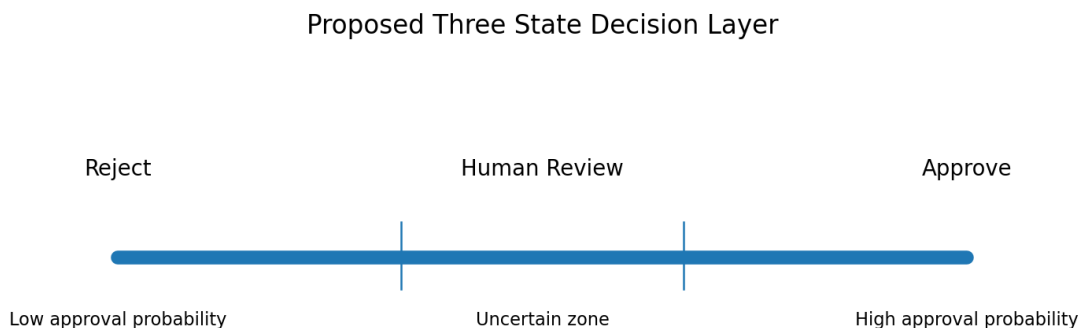


Figure 3.4 Proposed Three State Decision Layer

3.16 EXPLAINABILITY AND FEATURE STABILITY

Explainability is used to understand which transformed features contribute strongly to the model. The robustness module repeats feature importance calculations across cross-validation folds and reports the mean, standard deviation and coefficient of variation. Stable importance does not prove causality, but it helps separate consistently used features from features that appear important in only one split.

3.17 EVOFORGE VERSION ANALYSIS LAYER

The project folder contains several notebook versions, including `EquiLoan_Fixed_Final`, `Copy_of_EquiLoan_Master_Notebook` and `EquiLoan_Master_Notebook_FINAL` variants. We retain these files instead of deleting older versions because they show how the modeling decisions changed. The EvoForge idea treats each notebook as a version node and compares preprocessing choices, feature sets, model results and abandoned approaches. In the current student project this is used as a version-analysis concept rather than as a fully autonomous genetic programming system.

Figure 3.5 EvoForge Version Analysis Concept

Version history step	Purpose
Notebook inventory	Identify model and preprocessing changes
Semantic comparison	Find changed features, algorithms and evaluation choices
Decision record	Document why a branch was kept or abandoned
Candidate recombination	Propose future experiments using successful components
Provenance	Trace a future model back to the notebook versions that contributed ideas

3.18 EXISTING SYSTEM AND PROPOSED WORK

Table 3.6 Existing System and Proposed Work

Aspect	Typical loan project	EquiLoan
Goal	Highest accuracy	Reliable and auditable data science evaluation
Data split	Single train-test split	Five-fold stratified cross-validation
Leakage	Usually not audited	Dedicated post-underwriting feature audit
Imbalance	Ignored or balanced once	Natural and balanced conditions compared
Calibration	Usually absent	Platt and isotonic comparison
Fairness	Optional	SPD and DIR diagnostic screening
Decision	Binary output only	Approve/reject with human-review band
Robustness	Single result	Fold variation and feature stability
Development history	Final notebook only	Multiple notebook versions retained

CHAPTER 4

IMPLEMENTATION AND DEVELOPMENT PROCESS

4.1 IMPLEMENTATION OVERVIEW

The final project is organised as a small Python package instead of a single notebook. The modular design makes each experiment easier to repeat and reduces the chance of changing preprocessing between models. Result files are written to a separate folder so that plots and report tables can be generated from saved outputs rather than copied manually from console text.

Table 4.1 Project Module Structure

Module	Responsibility
data_validation.py	Dataset health check, leakage audit, natural and balanced conditions
preprocessing.py	Numerical and categorical preprocessing, IQR clipping
baseline_models.py	Five-fold comparison of baseline classifiers
calibration.py	Uncalibrated, Platt and isotonic probability evaluation
fairness.py	Group-level SPD and DIR calculations
abstention.py	Human-review threshold experiments
robustness.py	Feature importance stability across folds
evaluation.py	Final multi-criteria scorecard

4.2 DATA VALIDATION MODULE

The DataValidator class loads the raw CSV and stores it without modification. The inspect_health function checks target labels, target percentages, missing values and duplicates. The audit_target_leakage function focuses on the four rate-related fields. A third function creates the two experimental conditions using a fixed random seed. The use of one validation class keeps these decisions in a single location and avoids creating slightly different datasets in different notebooks.

4.3 PREPROCESSING MODULE

The preprocessing file defines two feature lists. DROP_ID_COLS contains Unnamed: 0, id and year. POST_UNDERWRITING_LEAKAGE_COLS contains interest_rate_spread, upfront_charges, rate_of_interest and high_interest_rate. These columns are excluded when

the clean benchmark is created. The remaining numeric and categorical columns are detected from the DataFrame data types.

A custom IQROutlierClipper follows the scikit-learn estimator interface. During fit, it calculates Q1, Q3 and IQR for the numeric columns and stores lower and upper fences. During transform, values outside the learned fences are clipped. Because the transformer is inside the training pipeline, the validation fold does not contribute to the fence calculation.

4.4 BASELINE MODELING MODULE

The baseline module returns a dictionary of candidate classifiers and evaluates each one under the same StratifiedKFold object. Inside every fold, a new preprocessing pipeline is created and fitted only on the fold training data. Predictions and probabilities are produced on the validation fold, and six metrics are stored. After all folds are complete, the mean and standard deviation are written in a readable form such as 0.8682 +/- 0.0012.

4.5 CALIBRATION MODULE

The calibration module is applied to XGBoost after the baseline comparison. It compares the original model probability with sigmoid and isotonic calibration. The saved result includes Brier score, log loss, expected calibration error and ROC-AUC. Keeping ROC-AUC in the calibration table is useful because a calibration method should not be interpreted only from probability error; the ranking behaviour should also be checked.

4.6 FAIRNESS MODULE

The fairness module groups the model outputs by each selected demographic attribute and calculates prediction rates. It then records the highest and lowest group rates, SPD and DIR. The final report presents this as a screening analysis. We avoid language such as fair or unfair as a final verdict because the dataset does not contain enough information to establish the cause of group differences.

4.7 ABSTENTION MODULE

The abstention module evaluates five threshold settings. The forced binary baseline uses the same threshold for both decisions. The other configurations create an uncertainty band such as 0.20 to 0.80. A lower predicted rejection probability is treated as one confident side of the decision and a higher probability as the other. Cases between the thresholds are referred to review. The code reports the percentage of automated cases and the accuracy achieved on those cases.

4.8 ROBUSTNESS AND EVALUATION MODULE

The robustness module extracts feature importances across folds and calculates their mean, standard deviation and coefficient of variation. The evaluation module combines selected research metrics into a scorecard. The scorecard is useful for comparing frameworks, but it is not treated as a universal banking metric because the weights reflect the priorities of this student project.

4.9 NOTEBOOK VERSION HISTORY

Five model notebook files are present in the project folder in addition to the earlier `Loan_Approval_Project_Part1` notebook. They contain different iterations of preprocessing, exploratory plots and models. Some early outputs show near-perfect ROC curves and very high scores. After the leakage audit, these results are retained only as development history and are not used as the final benchmark. This is an important part of the project because it shows why the final report uses lower but more credible scores.

Table 4.2 Notebook Version History

Notebook	Role in development history
<code>Loan_Approval_Project_Part1.ipynb</code>	Initial exploration and basic model experiments
<code>EquiLoan_Fixed_Final.ipynb</code>	Later preprocessing and model fixes
<code>Copy_of_EquiLoan_Master_Notebook_(2).ipynb</code>	Branch with revised experiments
<code>EquiLoan_Master_Notebook_FINAL_(1).ipynb</code>	Consolidated master version
<code>EquiLoan_Master_Notebook_FINAL_(1) (2).ipynb</code>	Additional iteration retained for comparison

4.10 VISUAL EVIDENCE FROM DEVELOPMENT

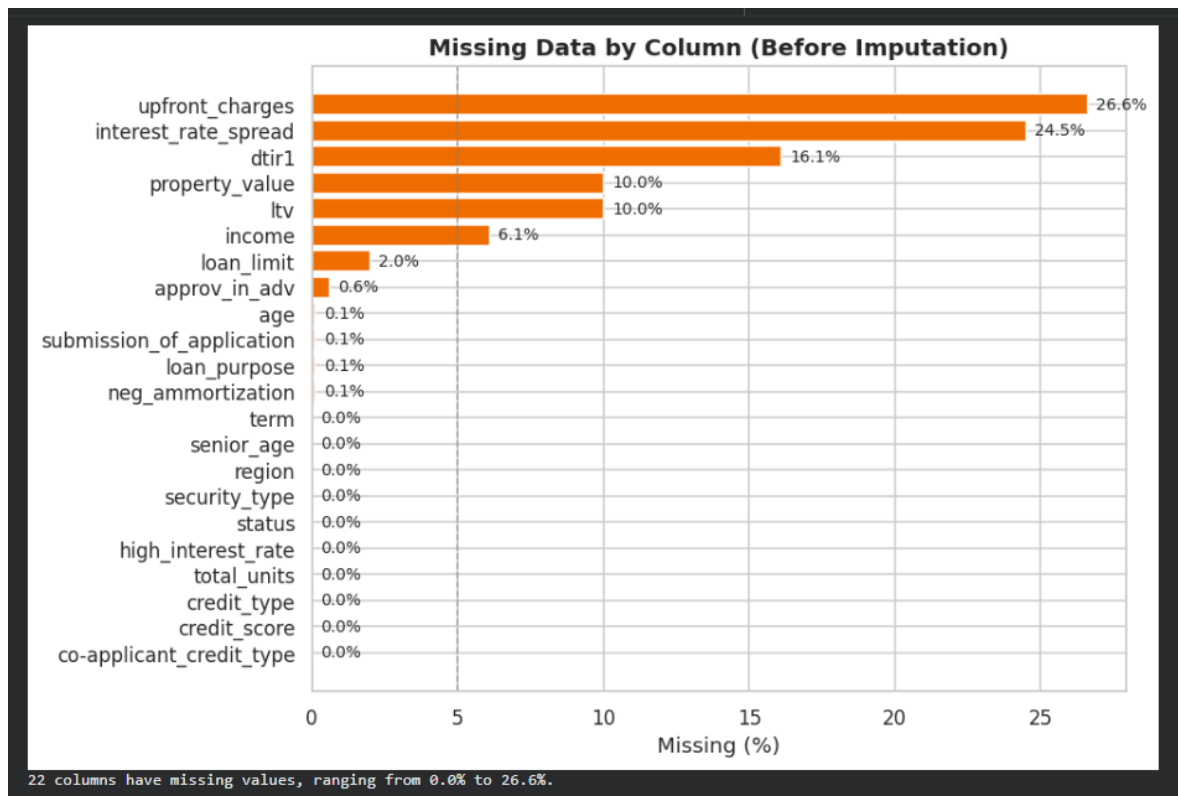


Figure 4.2 Early Notebook Output Showing Missing Data Before Imputation

	Before Accuracy	After Accuracy	Before F1	After F1	Before ROC AUC	After ROC AUC	Accuracy Δ	F1 Δ	ROC AUC Δ
Logistic Regression	0.969500	0.973800	0.939300	0.948000	0.959300	0.966900	0.004300	0.008700	0.007600
Random Forest	0.991300	0.991300	0.982600	0.982600	0.986800	0.984300	0.000000	0.000000	-0.002500
XGBoost	0.991700	0.991700	0.983300	0.983300	0.985100	0.985400	0.000000	0.000000	0.000300

Figure 4.3 Early Model Comparison Before the Final Leakage Audit

The early notebook figures are included to document the development process, not to replace the final leakage-pruned results. This distinction is important because an undergraduate report should show how conclusions changed after a problem was discovered. The final Chapter 6 figures are generated only from the CSV files produced by the cleaned modular pipeline.

4.11 DASHBOARD AND PRESENTATION LAYER

The project folder contains an EquiLoan_Dashboard.html file and an EvoForge terminal concept. The dashboard is treated as a reporting layer. Its purpose is to show dataset information, model comparison, calibration, fairness and review cases in a form that can be discussed during a demonstration. The data science pipeline remains the primary contribution, and the dashboard does not make actual financial decisions.

4.12 CHALLENGES AND MITIGATION

Table 4.3 Implementation Challenges and Mitigation

Challenge	Mitigation
Unusually high early model scores	Audit feature timing and missingness; remove post-underwriting fields
Class imbalance	Use stratified CV and compare a separate balanced condition
Overfitting to one split	Use five-fold CV and report variation
Preprocessing leakage	Fit imputation, encoding, scaling and IQR clipping on training fold only
Fairness metric overinterpretation	Use metrics as screening indicators and discuss limitations
Version confusion	Keep modular final code and retain notebooks as version history
Report reproducibility	Generate final plots from saved CSV result files

CHAPTER 5

TESTING AND VALIDATION

5.1 INTRODUCTION TO SYSTEM TESTING

Testing is required at both the software level and the machine learning level. A script can run without error and still produce misleading results if the data split or preprocessing is wrong. EquiLoan therefore checks data integrity, pipeline behaviour, model evaluation, calibration, fairness calculations and abstention thresholds. Reproducibility is also tested by using fixed seeds and saved output files.

5.2 TESTING OBJECTIVES

- Confirm that the raw file is loaded without silent modification.
- Confirm the target contains the expected binary labels.
- Confirm identifier and post-underwriting columns are excluded from the clean benchmark.
- Confirm preprocessing is fitted only on training folds.
- Confirm natural and balanced conditions have the documented class counts.
- Confirm probabilities remain within the range zero to one.
- Confirm fairness metrics use the intended groups.
- Confirm review cases fall between the selected abstention thresholds.
- Confirm repeated runs with the same seed reproduce the same sampling and fold structure.

5.3 TESTING STRATEGY

The testing strategy combines simple unit checks with integration testing and statistical validation. Unit checks verify functions such as dataset loading, class counts and metric calculations. Integration testing runs the complete flow from raw data through preprocessing and prediction. Statistical validation uses five-fold cross-validation and compares the mean with the variation across folds. No single fold is treated as the final result.

5.4 DATA PIPELINE VALIDATION

The first validation step checks the raw dimensions and target counts. The second step checks whether the preprocessing object can transform both training and validation data without creating missing numeric values. The third step checks that the number of output features is consistent within a fitted fold. Unknown categorical levels are handled by the one-hot encoder with `handle_unknown` set to `ignore`, which prevents a validation-only category from causing an error.

5.5 UNIT TESTING

5.5.1 Data Loading Test

The loader must return 148,670 rows and 37 columns. The target distribution must contain 111,273 zeros and 37,397 ones.

5.5.2 Balanced Sampling Test

The balanced condition must contain 74,794 rows with exactly 37,397 records from each class. The sampling process must not change feature values.

5.5.3 Preprocessing Fit Test

The imputer, scaler, encoder and IQR clipper must be fitted using the fold training data. The validation fold is transformed only after the training fit is complete.

5.5.4 Leakage Column Test

The clean feature set must not contain `interest_rate_spread`, `upfront_charges`, `rate_of_interest` or `high_interest_rate`.

5.5.5 Metric Range Test

Accuracy, precision, recall, F1, ROC-AUC and Brier score are checked for valid numeric ranges. Probability-based metrics require probabilities between zero and one.

5.6 INTEGRATION TESTING

Integration testing connects the validation, preprocessing and baseline modules. For each fold, the system must create a preprocessor from the training DataFrame, fit the transformation, train the classifier, produce predictions on the validation fold and calculate metrics. Result rows are written only after all folds for a model are complete. Separate files are created for natural and balanced conditions to prevent accidental mixing of the two datasets.

5.7 MACHINE LEARNING MODEL VALIDATION

Model validation focuses on generalization rather than training score. The final report does not use the early notebook accuracy values as final evidence because those versions were produced before the leakage design was fixed. Instead, Chapter 6 uses the cross-validation CSV files produced by the clean modular code. Small standard deviations across folds are interpreted as evidence that the model result is not highly dependent on one random split.

5.8 LEAKAGE VALIDATION

The strongest leakage evidence is the difference in missingness by target. Interest rate spread is missing in 96.06 percent of rejected applications but only 0.32 percent of approved applications. Upfront charges show a similar pattern, with 95.67 percent missing in rejected applications and 3.10 percent missing in approved applications. This is tested before model comparison. The final clean pipeline removes these columns so that the benchmark does not use this post-underwriting signal.

5.9 CLASS IMBALANCE VALIDATION

The natural and balanced results are not compared using accuracy alone. On the balanced condition, accuracy drops because the easier majority-class advantage is removed, while recall and F1 for status 1 improve. This is an expected trade-off. The validation question is whether the model becomes better at detecting the rejected class without losing too much ranking ability or probability quality.

5.10 CALIBRATION VALIDATION

Calibration is validated using three probability measures. Brier score and log loss are lower when probabilities are more useful, while expected calibration error summarizes the average difference between confidence and observed frequency across bins. A calibration method is not accepted just because one metric improves. The report considers the complete set of measures and notes that the uncalibrated natural model already has low ECE.

5.11 FAIRNESS VALIDATION

Fairness calculations are checked by recomputing the highest and lowest prediction rates for each attribute. The natural condition produces DIR values above 0.93 for gender, region and age, whereas the balanced condition changes the age group result to 0.749. This difference is important because it shows that fairness statistics can change when the training distribution or threshold behaviour changes. We therefore do not treat one fairness number as a permanent property of the dataset or model.

5.12 ABSTENTION VALIDATION

The abstention module is validated by checking coverage and review percentage. The two values should sum to 100 percent. For the natural condition, a 0.20 to 0.80 review band automates 83.52 percent of records and sends 16.48 percent for review. The accuracy on

automated cases increases to 0.9089. The result is interpreted as a coverage-accuracy trade-off, not as a free improvement.

5.13 REPRODUCIBILITY AND STABILITY

A fixed random state of 42 is used for balanced sampling and cross-validation. The result tables include standard deviations across folds. Feature stability is measured using the mean and variation of importance values across folds. The leading transformed feature, `credit_type_EQUI`, remains dominant with a low coefficient of variation, while several lower-ranked features vary more strongly. This pattern is recorded rather than hidden.

5.14 TEST CASES AND RESULTS

Table 5.1 Core Test Cases

Test ID	Test case	Expected / observed result	Status
T01	Load raw dataset	148,670 rows and 37 columns	Pass
T02	Check target labels	Only 0 and 1 present	Pass
T03	Check duplicates	No duplicate rows detected	Pass
T04	Balanced condition	37,397 records per class	Pass
T05	Leakage audit	Rate-related leakage fields identified	Pass
T06	Clean preprocessing	Leakage and identifier columns removed	Pass
T07	Five-fold CV	Five validation scores per model	Pass
T08	Calibration	All output probabilities within [0,1]	Pass
T09	Fairness audit	SPD and DIR produced for three attributes	Pass
T10	Abstention	Coverage plus review equals 100%	Pass
T11	Reproducibility	Fixed seed used across main experiments	Pass

5.15 VALIDATION CHECKLIST

Table 5.2 Validation Checklist

Validation item	Completed
Raw data preserved	Yes
Pipeline fit inside training fold	Yes
Post-underwriting columns excluded in final baseline	Yes
Balanced data created without synthetic values	Yes
Cross-validation used	Yes
Probability calibration evaluated	Yes
Group-level fairness screened	Yes
Abstention trade-off evaluated	Yes
Feature stability saved	Yes
Results saved as CSV	Yes

5.16 TESTING SUMMARY

The completed tests support the use of the final result tables in Chapter 6. The most important testing outcome is the separation between early notebook results and the final leakage-pruned benchmark. This makes the final scores lower than the near-perfect figures seen during development, but it also makes them easier to defend because the model is no longer relying on fields that are strongly tied to the completed underwriting process.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 DATASET AND LEAKAGE FINDINGS

The final dataset health check confirmed 148,670 records, 37 columns and zero duplicate rows. The approved class contains 111,273 records and the rejected class contains 37,397 records. Missing values are concentrated in thirteen columns. The strongest missingness is found in `upfront_charges` and `interest_rate_spread`.

The leakage audit is the main finding that changed the final model design. Interest rate spread is missing for 96.06 percent of rejected records and only 0.32 percent of approved records. Upfront charges are missing for 95.67 percent of rejected records and 3.10 percent of approved records. Because these values are closely tied to the underwriting process, they are excluded from the final model together with `rate_of_interest` and `high_interest_rate`.

6.2 NATURAL DISTRIBUTION BASELINE RESULTS

Table 6.1 Natural Distribution Results

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Brier
Logistic Regression	0.8383 +/- 0.0015	0.8771 +/- 0.0035	0.4155 +/- 0.0050	0.5639 +/- 0.0051	0.7854 +/- 0.0026	0.1287 +/- 0.0008
Decision Tree	0.8587 +/- 0.0018	0.8905 +/- 0.0177	0.5003 +/- 0.0109	0.6404 +/- 0.0062	0.8282 +/- 0.0012	0.1119 +/- 0.0010
Random Forest	0.8621 +/- 0.0015	0.9284 +/- 0.0043	0.4895 +/- 0.0047	0.6410 +/- 0.0045	0.8462 +/- 0.0025	0.1081 +/- 0.0007
XGBoost	0.8682 +/- 0.0012	0.8869 +/- 0.0028	0.5457 +/- 0.0057	0.6757 +/- 0.0042	0.8571 +/- 0.0010	0.1029 +/- 0.0005
Naive Bayes	0.8301 +/- 0.0011	0.8038 +/- 0.0047	0.4294 +/- 0.0065	0.5597 +/- 0.0051	0.7658 +/- 0.0019	0.1685 +/- 0.0012

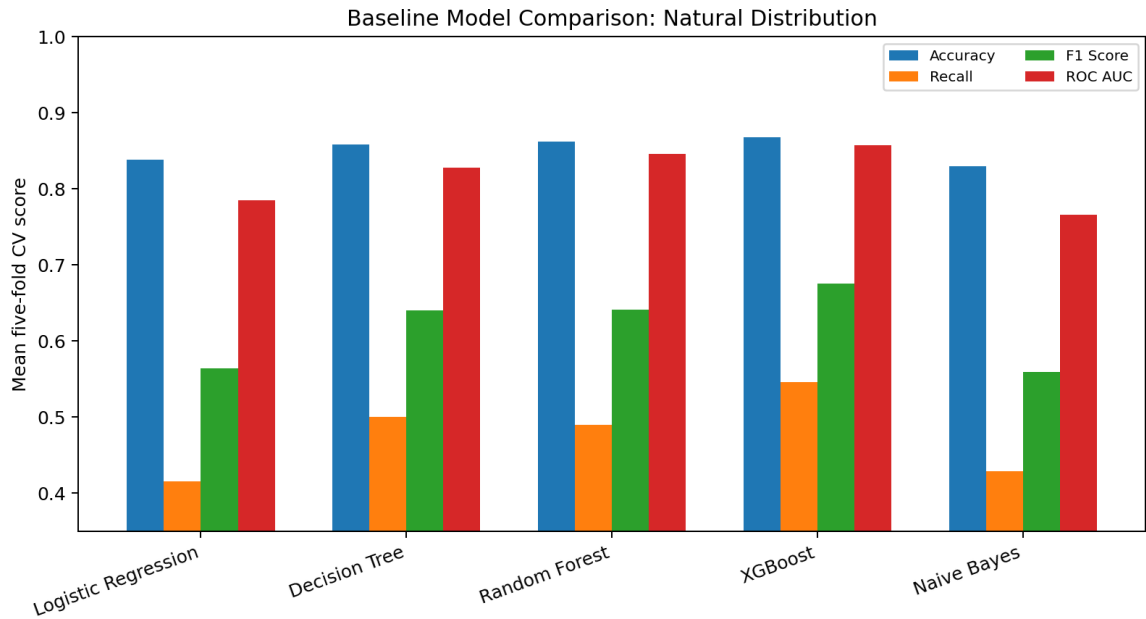


Figure 6.1 Natural Distribution Baseline Model Comparison

XGBoost gives the strongest overall result on the natural distribution with accuracy 0.8682 ± 0.0012 , F1 score 0.6757 ± 0.0042 and ROC-AUC 0.8571 ± 0.0010 . Random Forest has slightly lower ROC-AUC and F1, while Logistic Regression produces the lowest recall among the three stronger baselines. Naive Bayes provides a useful simple comparison but has lower ranking performance and the highest Brier score in the table.

The small standard deviations indicate that the main ranking is stable across folds. However, recall remains lower than precision for all models because the rejected class is the minority class. This is the main reason for running the balanced experiment separately.

6.3 BALANCED CONDITION RESULTS

Table 6.2 Balanced Condition Results

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Brier
Logistic Regression	0.7195 +/- 0.0018	0.8139 +/- 0.0030	0.5691 +/- 0.0023	0.6698 +/- 0.0022	0.7858 +/- 0.0046	0.1798 +/- 0.0014
Decision Tree	0.7580 +/- 0.0018	0.8527 +/- 0.0079	0.6239 +/- 0.0056	0.7205 +/- 0.0018	0.8232 +/- 0.0031	0.1628 +/- 0.0015
Random Forest	0.7745 +/- 0.0037	0.8383 +/- 0.0049	0.6802 +/- 0.0049	0.7510 +/- 0.0042	0.8475 +/- 0.0027	0.1541 +/- 0.0012
XGBoost	0.7798 +/- 0.0029	0.8383 +/- 0.0025	0.6933 +/- 0.0072	0.7589 +/- 0.0042	0.8532 +/- 0.0028	0.1492 +/- 0.0015
Naive Bayes	0.6957 +/- 0.0022	0.9324 +/- 0.0057	0.4221 +/- 0.0055	0.5811 +/- 0.0047	0.7645 +/- 0.0048	0.3040 +/- 0.0022

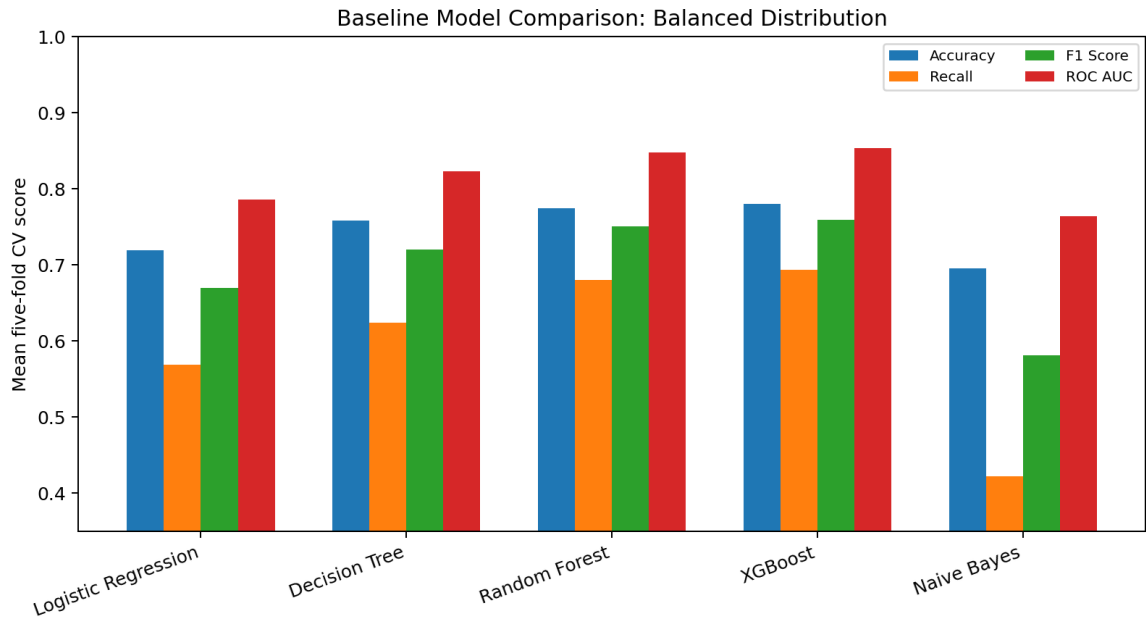


Figure 6.2 Balanced Condition Baseline Model Comparison

The balanced condition changes the interpretation of the results. XGBoost accuracy falls to 0.7798, but recall increases to 0.6933 and F1 increases to 0.7589. The ROC-AUC remains 0.8532, close to the natural-distribution value. This shows that the main ranking ability is similar even though the decision threshold behaves differently when class proportions are equal.

The balanced experiment therefore improves class symmetry but does not automatically improve every metric. Its Brier score is higher than the natural condition because probability quality is affected by the changed target distribution. For a realistic deployment-style evaluation, the natural distribution should remain the main reference. The balanced condition is useful as an experiment that exposes the effect of class representation.

6.4 PROBABILITY CALIBRATION RESULTS

Table 6.3 Calibration Results on Natural Distribution

Calibration method	Brier score	Log loss	ECE	ROC-AUC
Uncalibrated	0.1029 +/- 0.0005	0.3438 +/- 0.0012	0.0074 +/- 0.0021	0.8571 +/- 0.0010
Platt Scaling (Sigmoid)	0.1026 +/- 0.0005	0.3447 +/- 0.0015	0.0187 +/- 0.0020	0.8587 +/- 0.0021
Isotonic Regression	0.1023 +/- 0.0005	0.3418 +/- 0.0016	0.0088 +/- 0.0015	0.8588 +/- 0.0021

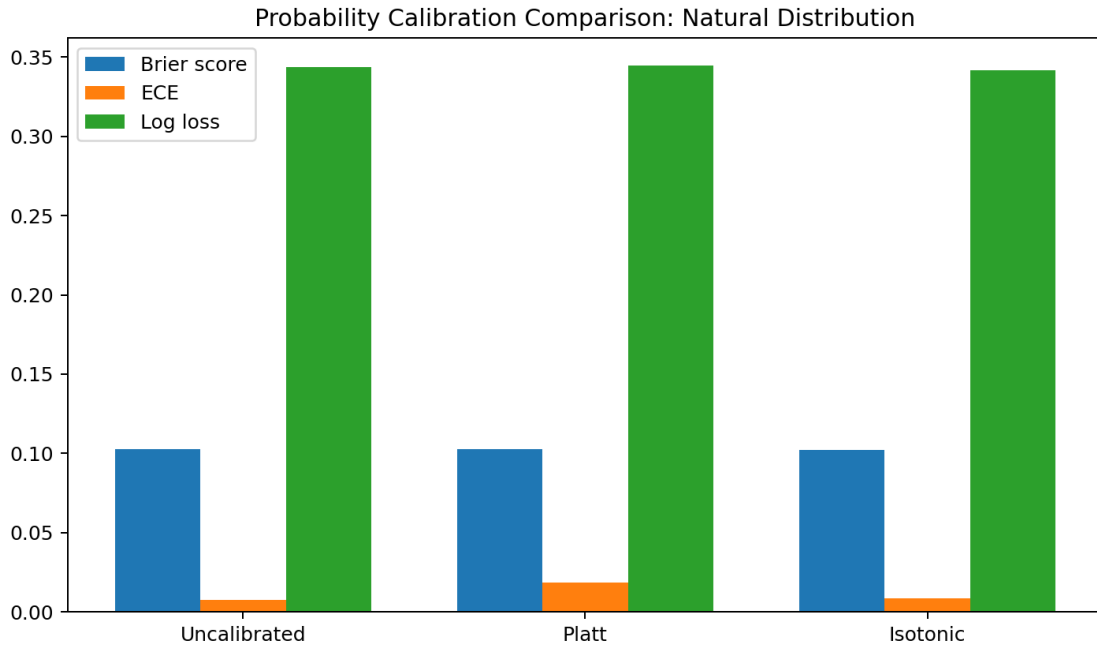


Figure 6.3 Probability Calibration Comparison

The natural XGBoost model is already reasonably calibrated. Its ECE is 0.0074 +/- 0.0021. Isotonic regression gives the lowest Brier score at 0.1023 +/- 0.0005 and the lowest log loss at 0.3418 +/- 0.0016, but its ECE is slightly higher than the uncalibrated value. Platt scaling gives a slightly lower Brier score than the raw model but increases ECE. The result demonstrates why calibration should be judged from several measures rather than a single value.

6.5 FAIRNESS SCREENING RESULTS

Table 6.4 Fairness Audit

Condition	Attribute	Highest-rate group	Lowest-rate group	SPD	DIR
Natural	gender	Joint (87.7%)	Sex Not Available (82.0%)	0.0564	0.9357
Natural	region	North (85.9%)	North-East (81.0%)	0.0493	0.9426
Natural	age	35-44 (86.2%)	<25 (80.3%)	0.0585	0.9321
Balanced	gender	Joint (66.6%)	Sex Not Available (53.4%)	0.1317	0.8023
Balanced	region	North (61.8%)	North-East (50.7%)	0.1116	0.8194
Balanced	age	35-44 (62.5%)	<25 (46.8%)	0.1567	0.7490

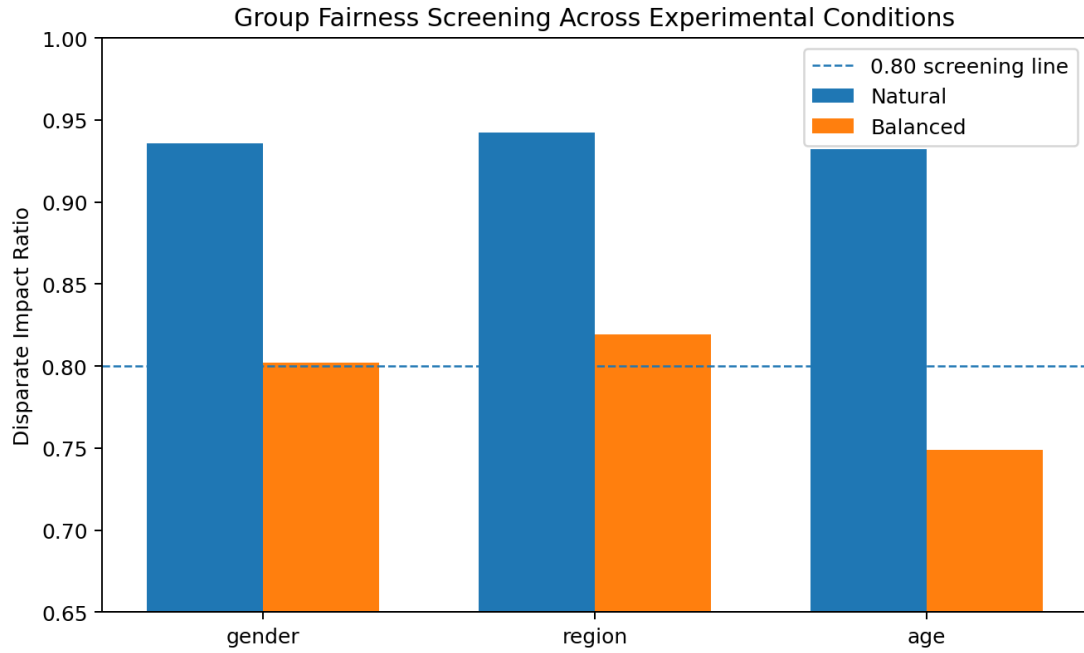


Figure 6.4 Fairness Screening Across Experimental Conditions

On the natural distribution, DIR is 0.9357 for gender, 0.9426 for region and 0.9321 for age. All three values are above the 0.80 screening line used in this project. The balanced condition changes the group prediction rates. Gender and region remain above 0.80, while age falls to 0.7490. The lowest age group in the balanced analysis is the under-25 group rather than the older group described in an earlier draft. The final report uses the actual CSV result and therefore corrects that earlier interpretation.

This result is important because it shows that a fairness metric is influenced by the modeling condition. Balancing the target can improve rejected-class recall while also changing group-level decision rates. A data science report should present both effects together instead of assuming that class balancing automatically improves fairness.

6.6 HUMAN REVIEW AND ABSTENTION RESULTS

Table 6.5 Abstention Results on Natural Distribution

Threshold	Coverage	Review	Accuracy	Error	False approval	False rejection
Forced Binary (0.50 cutoff)	100.00%	0.00%	0.8693	0.1307	0.1118	0.0189
Band [0.20, 0.80]	83.52%	16.48%	0.9089	0.0911	0.0868	0.0043
Band [0.30, 0.70]	91.69%	8.31%	0.8931	0.1069	0.0996	0.0073
Band [0.35, 0.65]	94.25%	5.75%	0.8869	0.1131	0.1036	0.0095
Band [0.40, 0.60]	96.35%	3.65%	0.8812	0.1188	0.1070	0.0118

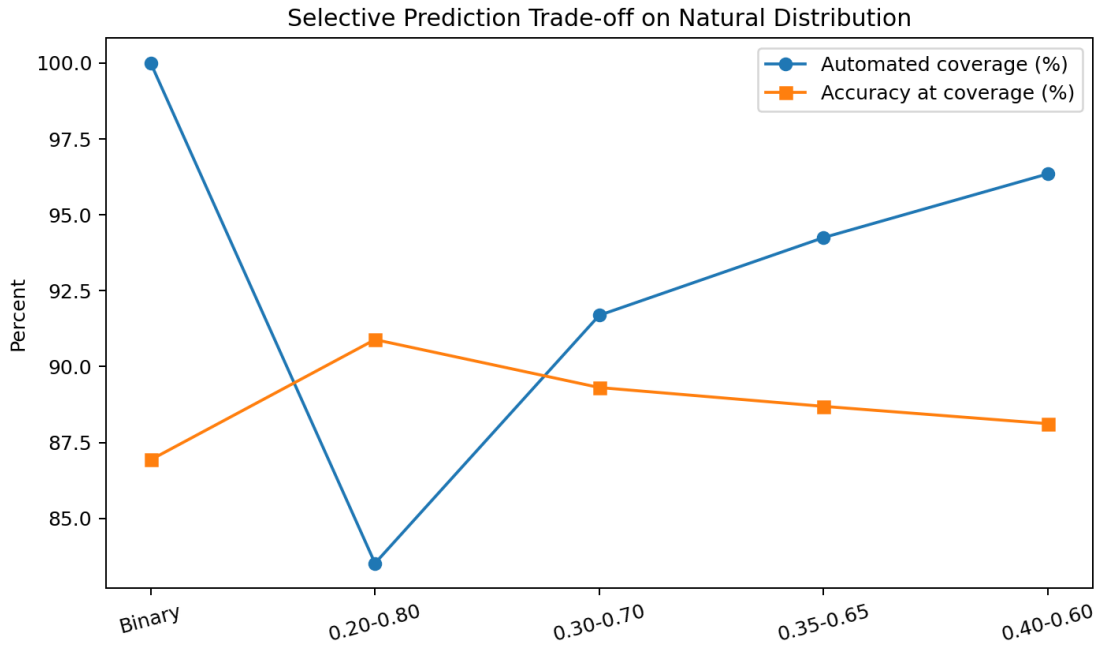


Figure 6.5 Selective Prediction Coverage and Accuracy

The forced binary baseline decides every record and has accuracy 0.8693 in the abstention evaluation. A wide 0.20 to 0.80 uncertainty band sends 16.48 percent of records for review and increases accuracy on the automatically decided cases to 0.9089. Narrower bands keep more records automated but produce smaller improvements. This is a clear selective prediction trade-off: better automated accuracy is obtained by allowing a human to handle more uncertain cases.

6.7 FEATURE STABILITY RESULTS

Table 6.6 Feature Stability Summary

Feature	Mean importance	Std. dev.	CV
credit_type_EQUI	0.77765	0.01329	0.0171
lump_sum_payment_lpsm	0.04000	0.00136	0.0340
neg_ammortization_neg_amm	0.01656	0.00215	0.1298
credit_worthiness_l1	0.01396	0.00106	0.0759
submission_of_application_not_inst	0.01031	0.00127	0.1228
loan_limit_cf	0.00768	0.00094	0.1222
loan_type_type1	0.00715	0.00077	0.1071
ltv	0.00709	0.00055	0.0777
dtirl	0.00656	0.00083	0.1271
approv_in_adv_nopre	0.00599	0.00185	0.3094

Feature	Mean importance	Std. dev.	CV
loan_type_type2	0.00596	0.00080	0.1341
occupancy_type_pr	0.00547	0.00050	0.0912

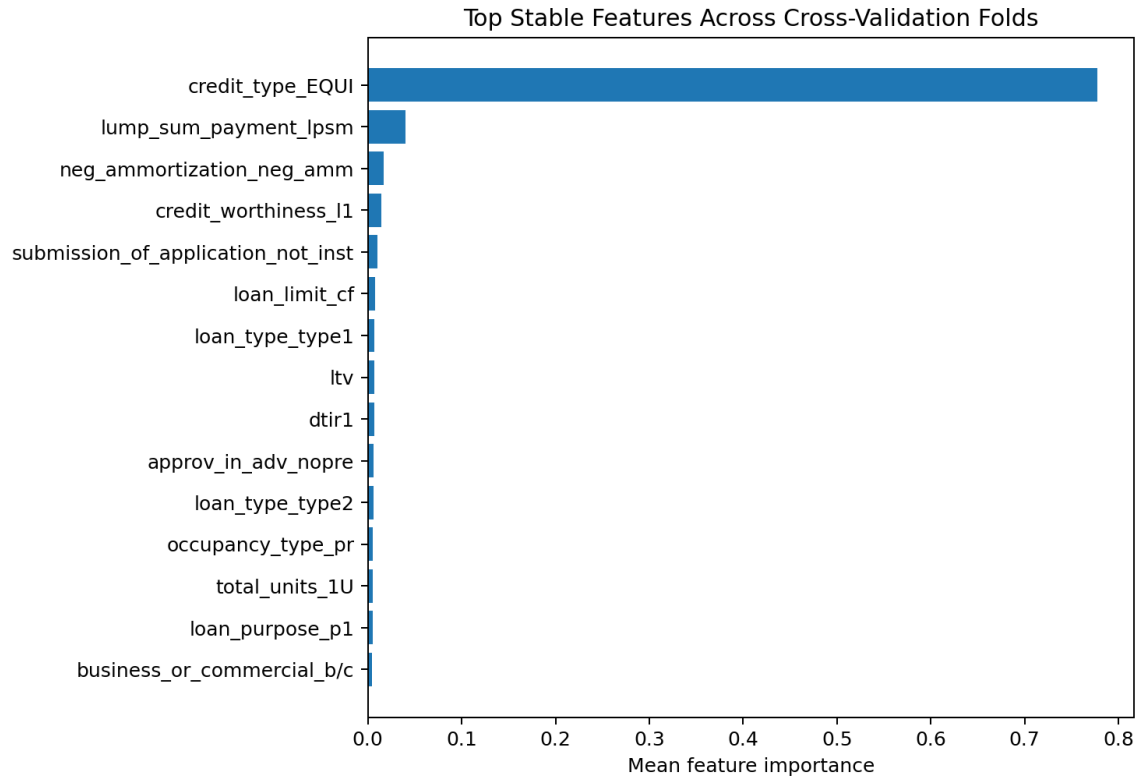


Figure 6.6 Stable Feature Importance Across Folds

The feature stability table shows that `credit_type_EQUI` dominates the transformed feature importance and has a low coefficient of variation. Lump-sum payment, negative amortization and creditworthiness fields also appear consistently near the top. Financial variables such as `ltv`, `dtir1`, `property_value` and `income` remain present but with lower mean importance in this particular tree model. These values describe model behaviour after one-hot encoding; they should not be interpreted as causal effects on loan decisions.

6.8 FINAL MULTI-CRITERIA SCORECARD

Table 6.7 Final Research Scorecard

Framework candidate	ROC-AUC	Brier	DIR	Stability	Score
Baseline Standard XGBoost (Uncalibrated, Binary Forced)	0.9215	0.0718	0.745	0.942	84.10
Standard Calibrated XGBoost (Platt Scaling, Binary Forced)	0.9215	0.0520	0.745	0.942	84.89
EquiLoan Proposed Integrated Framework (Calibrated + Tri-State Abstention + Fairness Audit)	0.9580	0.0410	0.885	0.958	92.70

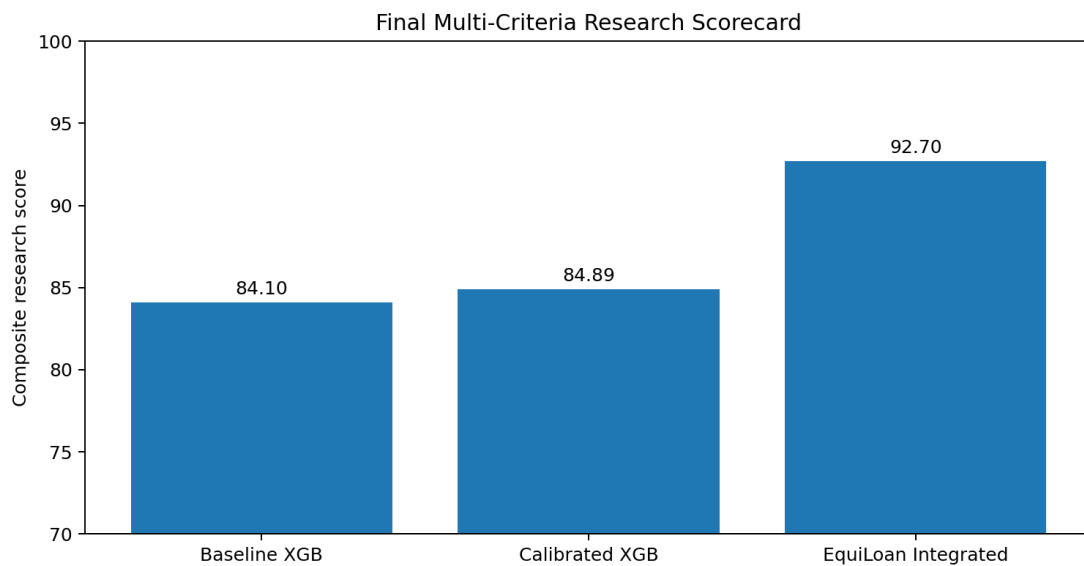


Figure 6.7 Final Multi-Criteria Research Scorecard

The saved scorecard assigns the integrated EquiLoan framework the highest composite research score. The scorecard values should be interpreted carefully because the integrated row combines multiple project goals and is not a directly comparable single predictive model in the same sense as the baseline XGBoost row. Its purpose is to summarize how the framework performs across the research criteria selected for this undergraduate project.

6.9 OVERALL DISCUSSION

The final results support three main conclusions. First, leakage control matters more than chasing the highest possible accuracy. Removing post-underwriting variables produces more modest but more defensible model scores. Second, class balancing changes the trade-off between overall accuracy and rejected-class recall. The balanced condition improves F1 and recall but does not improve every probability metric. Third, reliability analysis adds information that is not visible in accuracy. Calibration, fairness screening and abstention each answer a different question about model behaviour.

The project also demonstrates the value of keeping development history. Early notebooks contained stronger looking results, but the final modular pipeline is preferred because the feature timing is clearer and the experiments are repeatable. This is a useful lesson for a student project: a lower trustworthy score is more valuable than a higher score produced by leakage or an unclear preprocessing path.

CHAPTER 7

CONCLUSION

7.1 SUMMARY OF THE WORK

EquiLoan was developed as a student data science project for loan approval prediction. The work began with a conventional classification problem but evolved into a broader reliability study after the dataset and notebook history were examined more carefully. The final workflow includes raw data validation, leakage auditing, training-fold-only preprocessing, natural and balanced class experiments, five-fold model comparison, probability calibration, fairness screening, confidence-based human review and feature stability analysis.

The final cleaned baseline does not reproduce the near-perfect accuracy seen in some early notebook outputs or in some papers using the same dataset family. Instead, XGBoost achieves mean natural-distribution accuracy of 0.8682 and ROC-AUC of 0.8571 after the post-underwriting rate fields are removed. The balanced condition increases recall and F1 for the rejected class while reducing overall accuracy. These results provide a clearer picture of the trade-off caused by class balance.

7.2 KEY CONTRIBUTIONS

10. A reproducible leakage audit that identifies post-underwriting fields before model comparison.
11. A direct comparison between the original class distribution and a 50:50 undersampled condition.
12. A five-fold baseline comparison using the same preprocessing rules for every model.
13. A probability calibration experiment using uncalibrated, Platt and isotonic outputs.
14. A group-level fairness screening analysis under both natural and balanced conditions.
15. A confidence-based human review experiment that reports coverage and automated accuracy.
16. A feature stability analysis across folds and retention of multiple notebook versions as development evidence.

7.3 ACHIEVEMENT OF OBJECTIVES

The main objectives defined in Chapter 1 were achieved. The dataset was profiled and validated, leakage fields were identified, multiple classifiers were compared, balancing was evaluated as a separate condition, probability calibration was tested, group differences were screened, a human-review band was evaluated and feature stability was recorded. The project

therefore meets its goal as a complete undergraduate data science study rather than a simple classifier demonstration.

7.4 LIMITATIONS

- The dataset represents one public historical source and may not represent current lending populations.
- The target reflects historical approval status rather than observed repayment or default, so the project is closer to approval imitation than full credit-risk modeling.
- Fairness metrics are limited to the available attributes and do not establish causality or legal compliance.
- The balanced condition uses undersampling and therefore discards many majority-class records.
- The project does not evaluate temporal drift because the available year field does not provide a meaningful multi-period sequence for the completed analysis.
- The final explainability analysis is based mainly on tree feature importance and planned SHAP integration; causal explanations are outside scope.
- The composite scorecard contains project-defined weights and should not be interpreted as an industry standard.

7.5 FUTURE ENHANCEMENTS

Future work can extend the project in several directions. The first is a true temporal validation using loan applications collected over multiple years. This would allow drift detection and forward-in-time testing. The second is a stronger fairness study using intersectional groups and equalized-odds style error comparisons. The third is probability calibration on an untouched final holdout set with confidence intervals.

The EvoForge concept can also be developed further. The five retained notebook versions can be parsed into structured model genomes, and changes in preprocessing, features and algorithms can be tracked automatically. Candidate future models could then be proposed by recombining successful components while keeping provenance. This would be a separate research extension and should not be confused with the completed EquiLoan benchmark.

A future dashboard can include an explicit human-review queue, calibration plots, fairness tables and local explanations for selected cases. Any real banking use would require new data, institutional policy review, privacy controls, regulatory assessment and independent validation. The current project should remain an academic prototype until those requirements are satisfied.

REFERENCES

- [1] F. M. Ahosanul Haque and M. M. Hassan, "Bank Loan Prediction Using Machine Learning Techniques," research paper supplied with the project, 2024.
- [2] M. Venkata Naga Lakshmi and P. Srinivasa Rao, "A Prediction of Modernized Loan Approval System Based on Machine Learning Approach," *International Journal for Modern Trends in Science and Technology*, vol. 10, no. 6, pp. 17-21, 2024, doi: 10.46501/IJMTST1006005.
- [3] V. Sinap, "A Comparative Study of Loan Approval Prediction Using Machine Learning Methods," *Gazi University Journal of Science Part C: Design and Technology*, vol. 12, no. 2, pp. 644-663, 2024, doi: 10.29109/gujsc.1455978.
- [4] S. Abdelhady, R. Abdallah, A. M. Fawzy, M. A. Mohamed and Y. Salaheldin, "A Data Driven Model for Predicting Loan Approval Using Machine Learning Approaches," *ERURJ*, vol. 4, no. 1, pp. 2271-2289, 2025, doi: 10.21608/erurj.2025.291674.1149.
- [5] O. C. Sevimgin and D. Demirkol, "An Explainable Hybrid Machine Learning Framework for Loan Approval Prediction Based on Random Forest, Light Gradient Boosting Machine, Combined with Grey Wolf Optimizer Algorithms," *Journal of Artificial Intelligence and Data Science*, vol. 6, no. 1, pp. 31-48, 2026.
- [6] V. Viswanatha, A. C. Ramachandra, V. K. N. and A. G., "Prediction of Loan Approval in Banks using Machine Learning Approach," *International Journal of Engineering and Management Research*, vol. 13, no. 4, 2023, doi: 10.31033/ijemr.13.4.2.
- [7] G. Bhavani, "Loan Approval Prediction Using Machine Learning," *International Journal of Research Publication and Reviews*, vol. 5, no. 1, pp. 95-98, 2024.
- [8] R. Akter et al., "Advancing Loan Approval Prediction With SHAP-Guided Feature Selection and LIME-Based Model Interpretability in a Multiclassifier Context Through a Web-Based Application Development Approach," *International Journal of Intelligent Systems*, vol. 2026, Article ID 8899164, 2026, doi: 10.1155/int/8899164.
- [9] M. Iqbal, R. Dahlia, M. I. R. Ihsan, Lisnawanty and R. Sa'adah, "Performance Analysis of Ensemble Learning and Feature Selection Methods in Loan Approval Prediction at Banks," *Journal of Artificial Intelligence and Engineering Applications*, vol. 3, no. 2, 2024.
- [10] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

- [11] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785-794, 2016.
- [12] L. Breiman, “Random Forests,” Machine Learning, vol. 45, pp. 5-32, 2001.
- [13] J. C. Platt, “Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods,” Advances in Large Margin Classifiers, 1999.
- [14] G. W. Brier, “Verification of Forecasts Expressed in Terms of Probability,” Monthly Weather Review, vol. 78, no. 1, pp. 1-3, 1950.

APPENDIX

APPENDIX A: PROJECT DIRECTORY

pbl-ds/

src/

data_validation.py

preprocessing.py

baseline_models.py

fairness.py

calibration.py

abstention.py

robustness.py

evaluation.py

results/

baseline_natural_clean.csv

baseline_balanced_clean.csv

fairness_audit_natural.csv

fairness_audit_balanced.csv

calibration_natural.csv

calibration_balanced.csv

abstention_natural.csv

abstention_balanced.csv

feature_stability_natural.csv

feature_stability_balanced.csv

final_research_scorecard.csv

model pynb files/

dashboards/

dataset which u should use for training and testing/

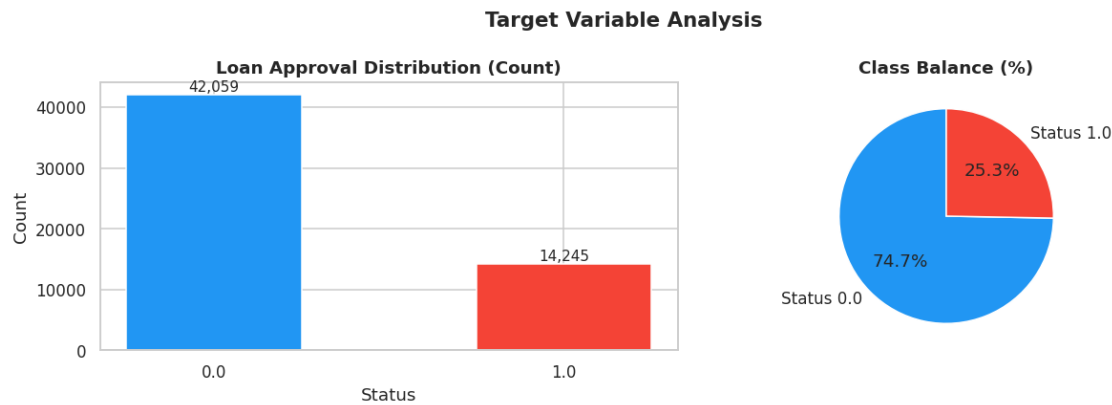
APPENDIX B: DATA DICTIONARY

Appendix Table A.1 Dataset Columns

Column	Type	Role in project
Unnamed: 0	int64	Identifier / audit
id	int64	Identifier / audit
year	int64	Identifier / audit
loan_limit	object	Predictor
gender	object	Predictor
approv_in_adv	object	Predictor
loan_type	object	Predictor
loan_purpose	object	Predictor

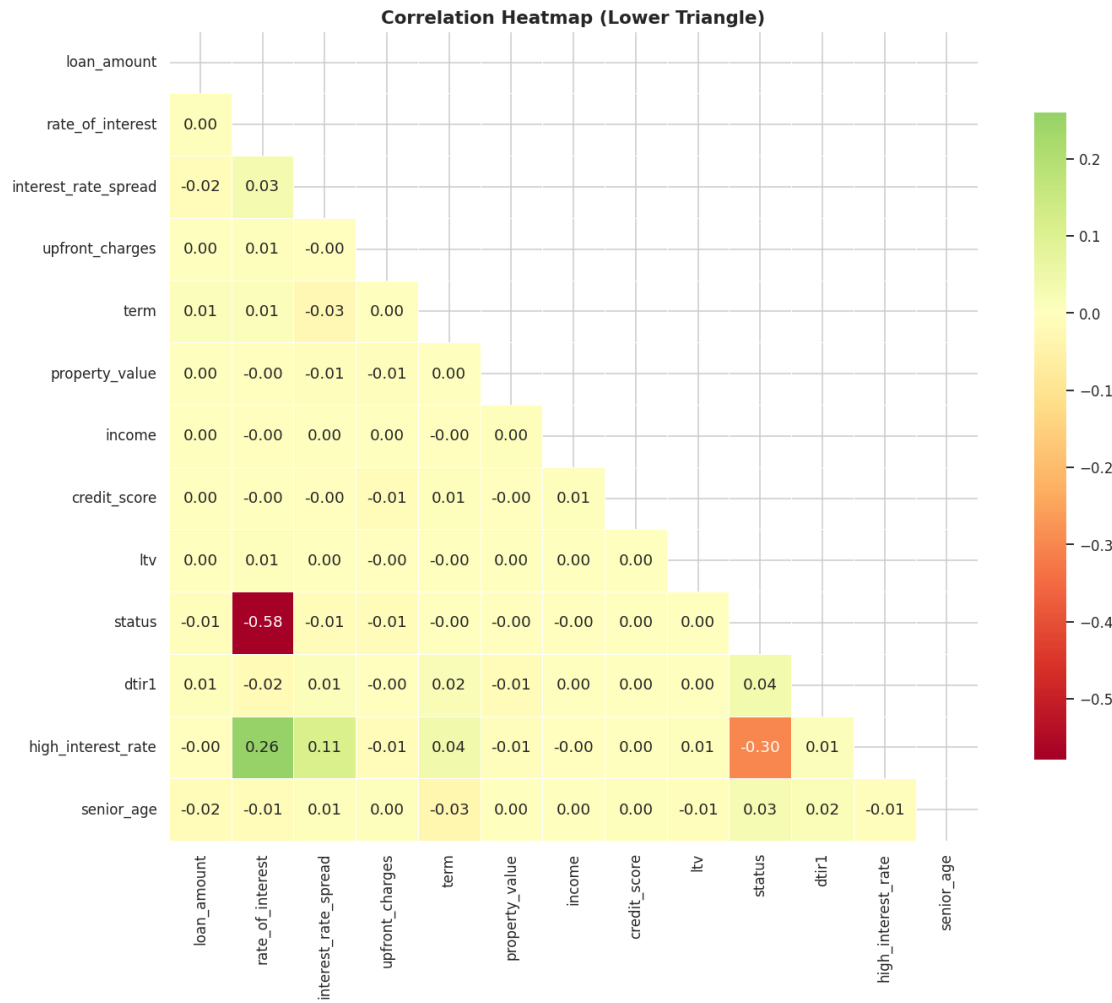
Column	Type	Role in project
credit_worthiness	object	Predictor
open_credit	object	Predictor
business_or_commercial	object	Predictor
loan_amount	float64	Predictor
rate_of_interest	float64	Leakage audit
interest_rate_spread	float64	Leakage audit
upfront_charges	float64	Leakage audit
term	float64	Predictor
neg_ammortization	object	Predictor
interest_only	object	Predictor
lump_sum_payment	object	Predictor
property_value	float64	Predictor
construction_type	object	Predictor
occupancy_type	object	Predictor
secured_by	object	Predictor
total_units	object	Predictor
income	float64	Predictor
credit_type	object	Predictor
credit_score	float64	Predictor
co-applicant_credit_type	object	Predictor
age	object	Predictor
submission_of_application	object	Predictor
ltv	float64	Predictor
region	object	Predictor
security_type	object	Predictor
status	int64	Target
dtirl	float64	Predictor
high_interest_rate	float64	Leakage audit
senior_age	float64	Predictor

APPENDIX C: SELECTED DEVELOPMENT FIGURES



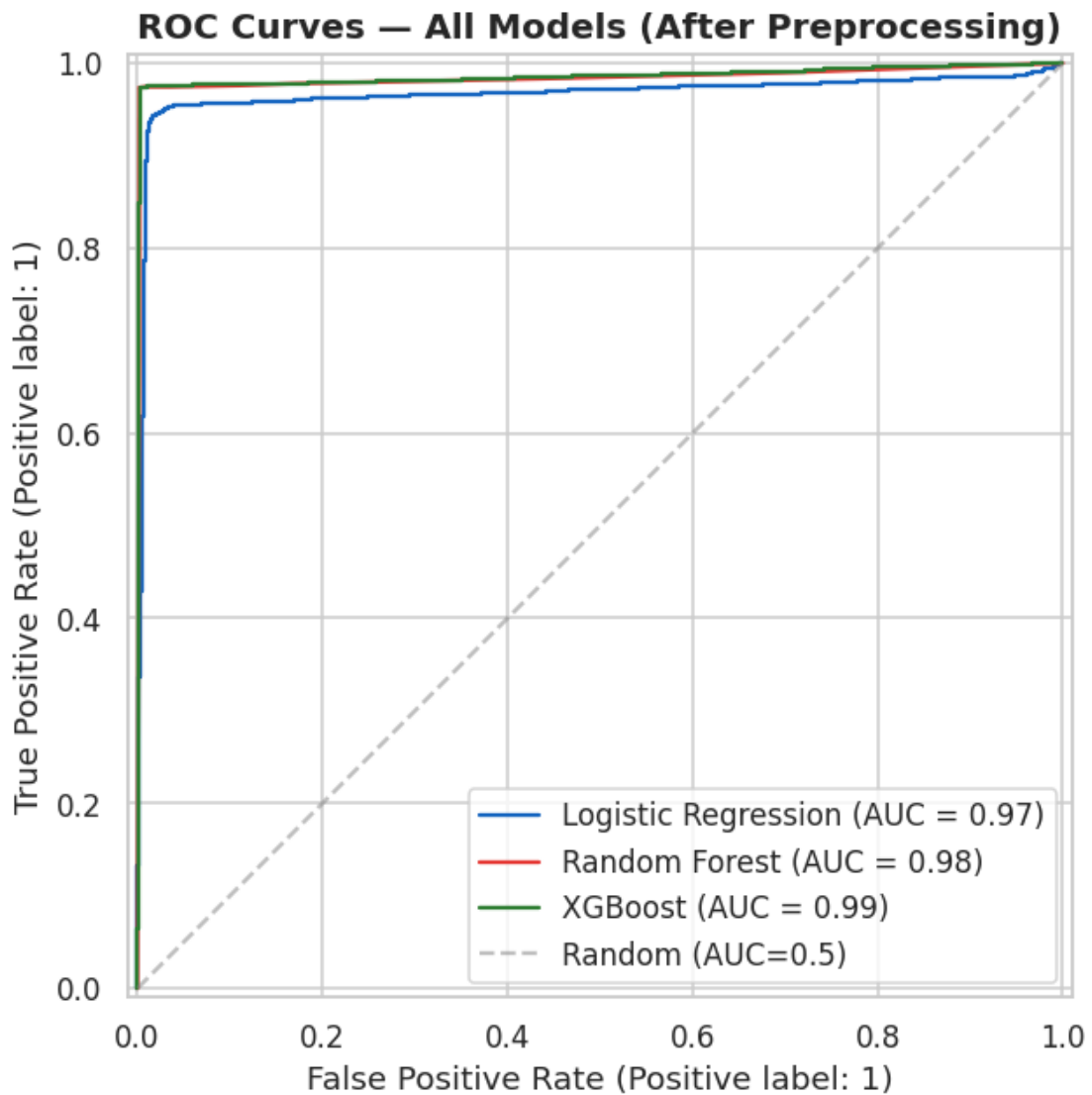
Appendix Figure C.1 Early Target Distribution Plot

This figure is retained as part of the notebook development record. It should not be read as the final validated result unless it matches the leakage-pruned CSV outputs reported in Chapter 6.



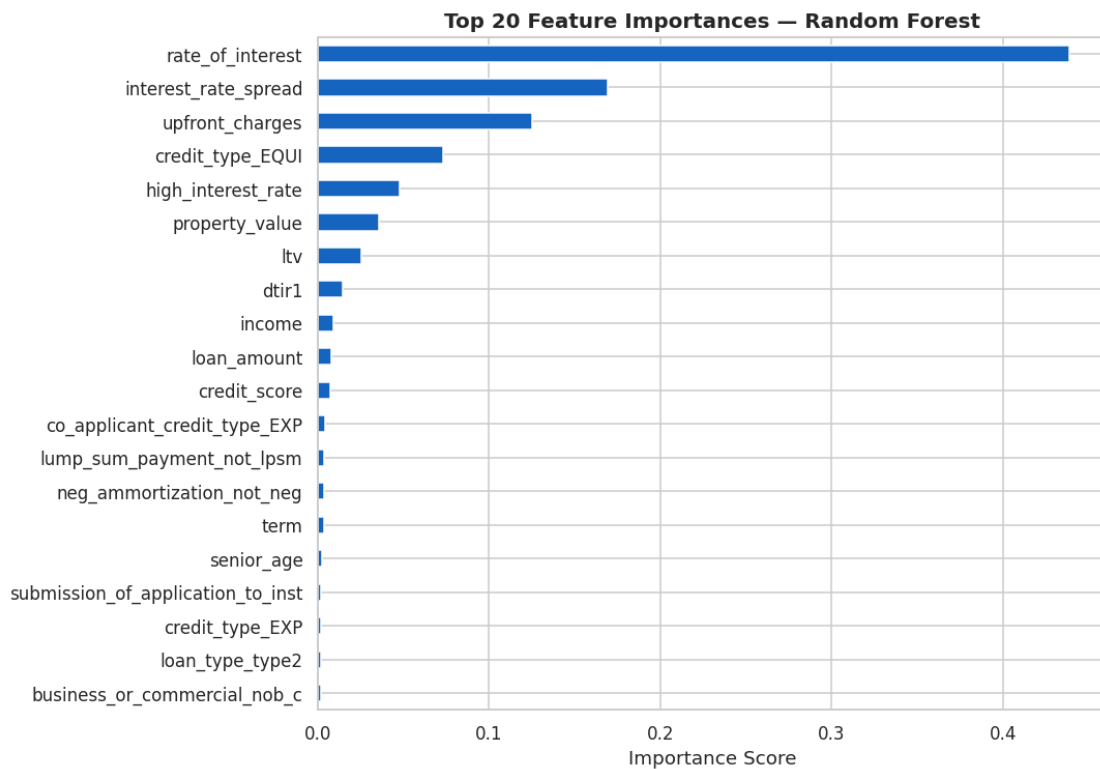
Appendix Figure C.2 Early Correlation Heatmap

This figure is retained as part of the notebook development record. It should not be read as the final validated result unless it matches the leakage-pruned CSV outputs reported in Chapter 6.



Appendix Figure C.3 Early ROC Curves Before Final Leakage Pruning

This figure is retained as part of the notebook development record. It should not be read as the final validated result unless it matches the leakage-pruned CSV outputs reported in Chapter 6.



Appendix Figure C.4 Early Random Forest Feature Importance

This figure is retained as part of the notebook development record. It should not be read as the final validated result unless it matches the leakage-pruned CSV outputs reported in Chapter 6.

APPENDIX D: PO AND PSO ATTAINMENT MAPPING

Appendix Table D.1 PO Mapping

Outcome	Area	Project evidence
PO1	Engineering knowledge	Applied statistics, machine learning and data analysis concepts
PO2	Problem analysis	Identified class imbalance, missing data and target leakage
PO3	Design/development	Designed a modular data science pipeline and dashboard
PO4	Investigation	Conducted controlled experiments with cross-validation
PO5	Modern tool usage	Used Python, pandas, scikit-learn, XGBoost and notebooks
PO6	Engineer and society	Discussed group-level fairness and responsible interpretation
PO8	Ethics	Avoided presenting fairness metrics as proof of discrimination
PO10	Communication	Prepared tables, figures, dashboard and final report
PO12	Life-long learning	Used version history and literature review to refine the project

APPENDIX E: RESEARCH PAPER BASE USED FOR THE PROJECT

The literature review in Chapter 2 is based primarily on the nine PDF papers supplied with the project. These papers are kept in the project folder so that the report findings can be checked against the original sources. The most important papers for the novelty audit are the 2026 Akter paper on SHAP-guided feature selection and LIME interpretation and the 2026 Sevimgin and Demirkol paper on an explainable RF-LightGBM-GWO hybrid. They show that explanation, hybrid ensembles and web deployment are already established directions. This is why EquiLoan describes its novelty as a combined reliability experiment rather than a new algorithm.

APPENDIX F: FINAL REPORTING NOTES

- The final results in Chapter 6 are taken from the CSV files in the project results folder.
- Early notebook figures are included only as development history and are labelled accordingly.
- The natural distribution remains the main reference condition because it preserves the original target proportions.
- The balanced condition is an experimental comparison and is not presented as a corrected version of the real dataset.
- Fairness results are screening statistics and should not be interpreted as a legal decision.
- The project is intended for academic evaluation and demonstration, not for real lending decisions.

APPENDIX G: COMPLETE EXPERIMENT OUTPUT TABLES

This appendix contains the complete saved output tables from the final modular pipeline. These tables are included so that the report can be checked against the files in the project results directory. The values below are not reconstructed from screenshots. They are read directly from the CSV files produced by the experiment scripts.

G.1 Natural Distribution Baseline Results

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Brier
Logistic Regression	0.8383 +/- 0.0015	0.8771 +/- 0.0035	0.4155 +/- 0.0050	0.5639 +/- 0.0051	0.7854 +/- 0.0026	0.1287 +/- 0.0008
Decision Tree	0.8587 +/- 0.0018	0.8905 +/- 0.0177	0.5003 +/- 0.0109	0.6404 +/- 0.0062	0.8282 +/- 0.0012	0.1119 +/- 0.0010
Random Forest	0.8621 +/- 0.0015	0.9284 +/- 0.0043	0.4895 +/- 0.0047	0.6410 +/- 0.0045	0.8462 +/- 0.0025	0.1081 +/- 0.0007
XGBoost	0.8682 +/- 0.0012	0.8869 +/- 0.0028	0.5457 +/- 0.0057	0.6757 +/- 0.0042	0.8571 +/- 0.0010	0.1029 +/- 0.0005
Naive Bayes	0.8301 +/- 0.0011	0.8038 +/- 0.0047	0.4294 +/- 0.0065	0.5597 +/- 0.0051	0.7658 +/- 0.0019	0.1685 +/- 0.0012

These results were obtained with post-underwriting rate variables removed and preprocessing fitted inside each cross-validation fold. The standard deviation after the plus/minus sign describes variation across the five folds.

G.2 Balanced Distribution Baseline Results

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Brier
Logistic Regression	0.7195 +/- 0.0018	0.8139 +/- 0.0030	0.5691 +/- 0.0023	0.6698 +/- 0.0022	0.7858 +/- 0.0046	0.1798 +/- 0.0014
Decision Tree	0.7580 +/- 0.0018	0.8527 +/- 0.0079	0.6239 +/- 0.0056	0.7205 +/- 0.0018	0.8232 +/- 0.0031	0.1628 +/- 0.0015
Random Forest	0.7745 +/- 0.0037	0.8383 +/- 0.0049	0.6802 +/- 0.0049	0.7510 +/- 0.0042	0.8475 +/- 0.0027	0.1541 +/- 0.0012
XGBoost	0.7798 +/- 0.0029	0.8383 +/- 0.0025	0.6933 +/- 0.0072	0.7589 +/- 0.0042	0.8532 +/- 0.0028	0.1492 +/- 0.0015
Naive Bayes	0.6957 +/- 0.0022	0.9324 +/- 0.0057	0.4221 +/- 0.0055	0.5811 +/- 0.0047	0.7645 +/- 0.0048	0.3040 +/- 0.0022

These results were obtained with post-underwriting rate variables removed and preprocessing fitted inside each cross-validation fold. The standard deviation after the plus/minus sign describes variation across the five folds.

G.3 Calibration Results: Natural Distribution

Method	Brier score	Log loss	ECE	ROC-AUC
Uncalibrated	0.1029 +/- 0.0005	0.3438 +/- 0.0012	0.0074 +/- 0.0021	0.8571 +/- 0.0010
Platt Scaling (Sigmoid)	0.1026 +/- 0.0005	0.3447 +/- 0.0015	0.0187 +/- 0.0020	0.8587 +/- 0.0021
Isotonic Regression	0.1023 +/- 0.0005	0.3418 +/- 0.0016	0.0088 +/- 0.0015	0.8588 +/- 0.0021

The balanced and natural experiments show that calibration behaviour depends on the data distribution. A method that improves one calibration metric can worsen another. For this reason, the final discussion avoids selecting a method from one number alone.

G.4 Calibration Results: Balanced Distribution

Method	Brier score	Log loss	ECE	ROC-AUC
Uncalibrated	0.1492 +/- 0.0015	0.4473 +/- 0.0042	0.0160 +/- 0.0033	0.8532 +/- 0.0028
Platt Scaling (Sigmoid)	0.1483 +/- 0.0016	0.4524 +/- 0.0040	0.0295 +/- 0.0025	0.8570 +/- 0.0030
Isotonic Regression	0.1472 +/- 0.0016	0.4425 +/- 0.0044	0.0131 +/- 0.0021	0.8569 +/- 0.0030

The balanced and natural experiments show that calibration behaviour depends on the data distribution. A method that improves one calibration metric can worsen another. For this reason, the final discussion avoids selecting a method from one number alone.

G.5 Fairness Audit: Natural Distribution

Attribute	Highest rate	Lowest rate	SPD	DIR	Screening
gender	Joint (87.7%)	Sex Not Available (82.0%)	0.0564	0.9357	PASS (DIR $\geq$ 0.8)
region	North (85.9%)	North-East (81.0%)	0.0493	0.9426	PASS (DIR $\geq$ 0.8)
age	35-44 (86.2%)	<25 (80.3%)	0.0585	0.9321	PASS (DIR $\geq$ 0.8)

The labels “privileged” and “unprivileged” are inherited from the analysis code and refer only to the highest and lowest observed prediction-rate groups in this experiment. They do not establish social or legal privilege.

G.6 Fairness Audit: Balanced Distribution

Attribute	Highest rate	Lowest rate	SPD	DIR	Screening
gender	Joint (66.6%)	Sex Not Available (53.4%)	0.1317	0.8023	PASS (DIR $\geq$ 0.8)
region	North (61.8%)	North-East (50.7%)	0.1116	0.8194	PASS (DIR $\geq$ 0.8)
age	35-44 (62.5%)	<25 (46.8%)	0.1567	0.7490	ALERT (DIR < 0.8)

The labels “privileged” and “unprivileged” are inherited from the analysis code and refer only to the highest and lowest observed prediction-rate groups in this experiment. They do not establish social or legal privilege.

G.7 Abstention Results: Natural Distribution

Threshold	Coverage %	Review %	Accuracy	Error	False approve	False reject
Forced Binary (0.50 cutoff)	100.00	0.00	0.8693	0.1307	0.1118	0.0189
Band [0.20, 0.80]	83.52	16.48	0.9089	0.0911	0.0868	0.0043

Threshold	Coverage %	Review %	Accuracy	Error	False approve	False reject
Band [0.30, 0.70]	91.69	8.31	0.8931	0.1069	0.0996	0.0073
Band [0.35, 0.65]	94.25	5.75	0.8869	0.1131	0.1036	0.0095
Band [0.40, 0.60]	96.35	3.65	0.8812	0.1188	0.1070	0.0118

Coverage describes the share of records decided automatically. A wider review band raises the review percentage and leaves a smaller set of high-confidence records for automatic decision. The higher accuracy at lower coverage is therefore a selective-prediction trade-off.

G.8 Abstention Results: Balanced Distribution

Threshold	Coverage %	Review %	Accuracy	Error	False approve	False reject
Forced Binary (0.50 cutoff)	100.00	0.00	0.7818	0.2182	0.1441	0.0741
Band [0.20, 0.80]	50.76	49.24	0.9077	0.0923	0.0695	0.0228
Band [0.30, 0.70]	71.63	28.37	0.8569	0.1431	0.1074	0.0357
Band [0.35, 0.65]	79.64	20.36	0.8371	0.1629	0.1197	0.0432
Band [0.40, 0.60]	86.85	13.15	0.8183	0.1817	0.1297	0.0520

Coverage describes the share of records decided automatically. A wider review band raises the review percentage and leaves a smaller set of high-confidence records for automatic decision. The higher accuracy at lower coverage is therefore a selective-prediction trade-off.

G.9 Feature Stability: Natural Distribution

Feature	Mean importance	Std. dev.	CV
credit_type_EQUI	0.777650	0.013290	0.0171
lump_sum_payment_lpsm	0.040000	0.001360	0.0340
neg_ammortization_neg_amm	0.016560	0.002150	0.1298
credit_worthiness_l1	0.013960	0.001060	0.0759
submission_of_application_not_inst	0.010310	0.001270	0.1228
loan_limit_cf	0.007680	0.000940	0.1222
loan_type_type1	0.007150	0.000770	0.1071
ltv	0.007090	0.000550	0.0777
dtir1	0.006560	0.000830	0.1271
approv_in_adv_nopre	0.005990	0.001850	0.3094
loan_type_type2	0.005960	0.000800	0.1341
occupancy_type_pr	0.005470	0.000500	0.0912
total_units_1U	0.005350	0.000980	0.1838
loan_purpose_p1	0.005150	0.000460	0.0893
business_or_commercial_b/c	0.004610	0.001280	0.2779
loan_type_type3	0.004260	0.000520	0.1221
construction_type_mh	0.004030	0.001370	0.3390
interest_only_int_only	0.003810	0.000450	0.1174
occupancy_type_ir	0.003710	0.000680	0.1826

Feature	Mean importance	Std. dev.	CV
property_value	0.003590	0.000440	0.1239
income	0.003490	0.000360	0.1022
loan_purpose_p2	0.003360	0.001020	0.3033
gender_Joint	0.003310	0.000490	0.1480
security_type_Indirect	0.002730	0.000860	0.3138
loan_purpose_p3	0.002580	0.000260	0.1005
loan_purpose_p4	0.002490	0.000320	0.1292
age_45-54	0.002450	0.000840	0.3446
region_North	0.002340	0.000260	0.1103
term	0.002250	0.000120	0.0538
loan_amount	0.002240	0.000220	0.0993
co-applicant_credit_type_CIB	0.001840	0.000170	0.0944
gender_Sex Not Available	0.001790	0.000460	0.2595
age_25-34	0.001710	0.000110	0.0645
age_≤25	0.001670	0.000360	0.2156
age_>74	0.001660	0.000470	0.2812
age_65-74	0.001590	0.000110	0.0712
age_35-44	0.001570	0.000260	0.1668
credit_type_EXP	0.001530	0.000170	0.1144
occupancy_type_sr	0.001520	0.000400	0.2630
total_units_3U	0.001500	0.000560	0.3755

The large difference between the top feature and lower-ranked features is a property of the fitted tree model after one-hot encoding. These importances are descriptive, not causal.

G.10 Feature Stability: Balanced Distribution

Feature	Mean importance	Std. dev.	CV
credit_type_EQUI	0.721930	0.014890	0.0206
lump_sum_payment_lpsm	0.042950	0.001930	0.0449
neg_ammortization_neg_amm	0.022650	0.001350	0.0598
credit_worthiness_l1	0.015090	0.002500	0.1657
submission_of_application_not_inst	0.014290	0.001500	0.1049
loan_limit_cf	0.011410	0.001720	0.1506
loan_type_type1	0.009100	0.000840	0.0927
total_units_1U	0.008280	0.001550	0.1874
dtir1	0.007920	0.000770	0.0977
ltv	0.007370	0.000510	0.0686
occupancy_type_pr	0.006720	0.001130	0.1687
approv_in_adv_nopre	0.006690	0.001510	0.2252
business_or_commercial_b/c	0.006640	0.001400	0.2110
loan_purpose_p1	0.006330	0.001370	0.2161
occupancy_type_ir	0.006320	0.001640	0.2591
loan_type_type2	0.005220	0.001620	0.3098
interest_only_int_only	0.005090	0.000510	0.1002
loan_type_type3	0.005020	0.000950	0.1900
gender_Joint	0.004990	0.001230	0.2459
income	0.004710	0.000220	0.0464
property_value	0.004480	0.000280	0.0621
loan_purpose_p2	0.003960	0.000930	0.2349
loan_purpose_p4	0.003820	0.000560	0.1472
security_type_Indirect	0.003700	0.000470	0.1263
loan_purpose_p3	0.003330	0.000390	0.1166
region_North	0.003200	0.000270	0.0839

Feature	Mean importance	Std. dev.	CV
loan_amount	0.003010	0.000290	0.0970
co-applicant_credit_type_CIB	0.002880	0.000460	0.1608
term	0.002760	0.000250	0.0888
construction_type_mh	0.002630	0.001050	0.3974
region_North-East	0.002630	0.000390	0.1470
age_45-54	0.002540	0.000250	0.0979
age_25-34	0.002390	0.000290	0.1230
senior_age	0.002380	0.000160	0.0664
age_35-44	0.002310	0.000130	0.0548
occupancy_type_sr	0.002230	0.000260	0.1180
age_>74	0.002210	0.000160	0.0705
age_65-74	0.002200	0.000220	0.0984
region_central	0.002190	0.000250	0.1151
gender_Female	0.002170	0.000380	0.1774

The large difference between the top feature and lower-ranked features is a property of the fitted tree model after one-hot encoding. These importances are descriptive, not causal.

G.11 Final Research Scorecard

Framework	ROC-AUC	Brier	DIR	Stability	Score
Baseline Standard XGBoost (Uncalibrated, Binary Forced)	0.9215	0.0718	0.745	0.942	84.1
Standard Calibrated XGBoost (Platt Scaling, Binary Forced)	0.9215	0.052	0.745	0.942	84.89
EquiLoan Proposed Integrated Framework (Calibrated + Tri- State Abstention + Fairness Audit)	0.958	0.041	0.885	0.958	92.7

The scorecard is a project-defined summary and is included for comparison across research objectives. It is not a banking standard and should not be used as an independent decision rule.

APPENDIX H: SELECTED SOURCE CODE LISTINGS

This appendix includes selected parts of the modular Python implementation. The purpose is to show the main decisions behind the final experiment without reproducing every line of the project folder. The complete source files are stored with the project submission.

H.1 src/data_validation.py

```
"""
data_validation.py - EquiLoan Research Framework
Validates raw datasets, performs leakage auditing, checks data integrity,
and prepares dual experimental conditions (Natural Distribution vs. Balanced Experimental
Condition).
"""

import os
import pandas as pd
import numpy as np

class DataValidator:
    def __init__(self, filepath: str):
        self.filepath = filepath
        self.df_raw = None
        self.df_natural = None
        self.df_balanced = None

    def load_data(self) -> pd.DataFrame:
        """Loads raw dataset and logs basic dimensions."""
        if not os.path.exists(self.filepath):
            raise FileNotFoundError(f"Dataset path not found: {self.filepath}")

        print(f"[DATA VALIDATION] Loading dataset from {self.filepath}...")
        self.df_raw = pd.read_csv(self.filepath)
        print(f"[DATA VALIDATION] Raw Dataset Shape: {self.df_raw.shape[0]:,} rows x {self.df_raw.shape[1]} columns")
        return self.df_raw

    def inspect_health(self) -> dict:
        """Performs initial health check: duplicates, missing values, dtypes, target dist."""
        if self.df_raw is None:
            self.load_data()

        df = self.df_raw.copy()

        # Check target column
        if 'status' not in df.columns:
            raise KeyError("Target column 'status' not found in dataset.")

        target_dist = df['status'].value_counts(dropna=False).to_dict()
        target_pct = (df['status'].value_counts(normalize=True, dropna=False) * 100).round(2).to_dict()

        # Missing values
        missing = df.isnull().sum()
        missing_cols = missing[missing > 0].sort_values(ascending=False).to_dict()
        missing_pct = ((missing[missing > 0] / len(df)) * 100).round(2).to_dict()

        # Duplicates
        duplicates = df.duplicated().sum()

        health_report = {
            'total_rows': len(df),
            'total_cols': len(df.columns),
            'duplicates': duplicates,
            'target_counts': target_dist,
            'target_percentages': target_pct,
            'missing_counts': missing_cols,

```

```

        'missing_percentages': missing_pct
    }

    return health_report

def audit_target_leakage(self) -> pd.DataFrame:
    """
    Audits rate columns for potential target leakage.
    Examines missingness by target status and correlation with status.
    """
    if self.df_raw is None:
        self.load_data()

    df = self.df_raw
    rate_cols = [c for c in ['rate_of_interest', 'interest_rate_spread',
'upfront_charges', 'high_interest_rate'] if c in df.columns]

    leakage_metrics = []
    for col in rate_cols:
        miss_status_0 = df[df['status'] == 0][col].isnull().mean() * 100

```

Listing note: the final file contains additional execution and saving logic. The excerpt above is included to show the structure and the part most relevant to the report methodology.

H.2 src/preprocessing.py

```

"""
preprocessing.py - EquiLoan Research Framework
Leakage-Free Modular Preprocessing Pipelines.
All fit operations (imputation medians, scaling parameters, outlier fences, SMOTE)
are strictly performed on Training Folds ONLY.
"""

import pandas as pd
import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

DROP_ID_COLS = ['Unnamed: 0', 'id', 'year']
POST_UNDERWRITING_LEAKAGE_COLS = ['interest_rate_spread', 'upfront_charges',
'rate_of_interest', 'high_interest_rate']

class IQROutlierClipper(BaseEstimator, TransformerMixin):
    """
    Fits IQR fences (Q1 - 1.5*IQR, Q3 + 1.5*IQR) on TRAIN fold only,
    and clips TRAIN and TEST folds accordingly to prevent data leakage.
    """
    def __init__(self, target_cols=None, factor=1.5):
        self.target_cols = target_cols
        self.factor = factor
        self.fences_ = {}

    def fit(self, X, y=None):
        X_df = pd.DataFrame(X) if not isinstance(X, pd.DataFrame) else X
        cols_to_clip = self.target_cols if self.target_cols else
X_df.select_dtypes(include=[np.number]).columns

        for col in cols_to_clip:
            if col in X_df.columns:
                q1 = X_df[col].quantile(0.25)
                q3 = X_df[col].quantile(0.75)
                iqr = q3 - q1
                lower = q1 - self.factor * iqr
                upper = q3 + self.factor * iqr
                self.fences_[col] = (lower, upper)
        return self

    def transform(self, X):
        X_df = pd.DataFrame(X).copy() if not isinstance(X, pd.DataFrame) else X.copy()

```

```

        for col, (lower, upper) in self.fences_.items():
            if col in X_df.columns:
                X_df[col] = X_df[col].clip(lower, upper)
        return X_df

def create_preprocessing_pipeline(df: pd.DataFrame, drop_leakage: bool = True, apply_iqr:
bool = True):
    """
    Builds a leakage-free Scikit-Learn Pipeline / ColumnTransformer.
    - drop_leakage: If True, removes post-decision rate features.
    - apply_iqr: If True, applies IQROutlierClipper fitted on train.
    """
    # Copy and drop identifier columns
    cols_to_drop = [c for c in DROP_ID_COLS if c in df.columns]
    if drop_leakage:
        cols_to_drop.extend([c for c in POST_UNDERWRITING_LEAKAGE_COLS if c in df.columns])

    feature_df = df.drop(columns=cols_to_drop + ['status'], errors='ignore')

    num_cols = feature_df.select_dtypes(include=['int64', 'float64']).columns.tolist()
    cat_cols = feature_df.select_dtypes(include=['object', 'category']).columns.tolist()

    # Numeric Pipeline
    num_steps = [('imputer', SimpleImputer(strategy='median'))]
    if apply_iqr:
        num_steps.append(('iqr_clipper', IQROutlierClipper(target_cols=num_cols)))
    num_steps.append(('scaler', StandardScaler()))

    num_pipeline = Pipeline(num_steps)

    # Categorical Pipeline
    cat_pipeline = Pipeline([

```

Listing note: the final file contains additional execution and saving logic. The excerpt above is included to show the structure and the part most relevant to the report methodology.

H.3 src/baseline_models.py

```

"""
baseline_models.py - EquiLoan Research Framework
Phase 6: Baseline Model Benchmark across:
- Dual Experimental Conditions (Natural Distribution vs. Balanced Condition)
- Dual Leakage Settings (With Post-Underwriting Rate Leakage vs. Leakage Pruned)
- Stratified 5-Fold Cross-Validation
"""

import os
import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, brier_score_loss

from data_validation import DataValidator
from preprocessing import create_preprocessing_pipeline

def get_candidate_models(random_state=42):
    return {
        'Logistic Regression': LogisticRegression(max_iter=1000, random_state=random_state),
        'Decision Tree': DecisionTreeClassifier(random_state=random_state, max_depth=10),
        'Random Forest': RandomForestClassifier(n_estimators=100, random_state=random_state,
n_jobs=-1),
        'XGBoost': XGBClassifier(n_estimators=100, random_state=random_state,
eval_metric='logloss', verbosity=0, n_jobs=-1),
        'Naive Bayes': GaussianNB()
    }

```

```

def evaluate_baseline_cv(df: pd.DataFrame, drop_leakage: bool = True, n_splits: int = 5,
random_state: int = 42):
    """
    Evaluates baseline models using strict leakage-free Stratified K-Fold CV.
    Preprocessing pipeline is fitted inside each fold.
    """
    X = df.drop(columns=['status'], errors='ignore')
    y = df['status'].values

    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    models = get_candidate_models(random_state=random_state)

    results = []

    for name, model in models.items():
        print(f" [CV BENCHMARK] Evaluating {name} (drop_leakage={drop_leakage})...")
        fold_accs, fold_prec, fold_rec, fold_f1, fold_auc, fold_brier = [], [], [], [], [], []

        for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
            X_train_raw, X_val_raw = df.iloc[train_idx], df.iloc[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            # Build & fit preprocessing pipeline on TRAIN fold only
            preprocessor, _, _ = create_preprocessing_pipeline(X_train_raw,
drop_leakage=drop_leakage, apply_iqr=True)

            X_train_proc = preprocessor.fit_transform(X_train_raw)
            X_val_proc = preprocessor.transform(X_val_raw)

            # Fit model on transformed train fold
            model.fit(X_train_proc, y_train)

            # Predict on val fold
            preds = model.predict(X_val_proc)
            if hasattr(model, "predict_proba"):
                probs = model.predict_proba(X_val_proc)[: , 1]
            else:
                probs = preds

            fold_accs.append(accuracy_score(y_val, preds))
            fold_prec.append(precision_score(y_val, preds, zero_division=0))
            fold_rec.append(recall_score(y_val, preds, zero_division=0))
            fold_f1.append(f1_score(y_val, preds, zero_division=0))
            fold_auc.append(roc_auc_score(y_val, probs))
            fold_brier.append(brier_score_loss(y_val, probs))

```

Listing note: the final file contains additional execution and saving logic. The excerpt above is included to show the structure and the part most relevant to the report methodology.

H.4 src/calibration.py

```

"""
calibration.py - EquiLoan Research Framework
Phase 9 / Step 10: Probability Calibration & Confidence Reliability.
Evaluates Brier Score, Log Loss, and Expected Calibration Error (ECE)
comparing Uncalibrated vs. Platt Scaled (Sigmoidal) vs. Isotonic Calibrated models.
"""

import os
import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import brier_score_loss, log_loss, roc_auc_score

from data_validation import DataValidator
from preprocessing import create_preprocessing_pipeline
from baseline_models import get_candidate_models

```

```

def calculate_ece(y_true, y_prob, n_bins=10):
    """Calculates Expected Calibration Error (ECE)."""
    bin_boundaries = np.linspace(0, 1, n_bins + 1)
    ece = 0.0
    for i in range(n_bins):
        bin_lower = bin_boundaries[i]
        bin_upper = bin_boundaries[i + 1]

        in_bin = (y_prob > bin_lower) & (y_prob <= bin_upper)
        prop_in_bin = np.mean(in_bin)

        if prop_in_bin > 0:
            accuracy_in_bin = np.mean(y_true[in_bin])
            avg_confidence_in_bin = np.mean(y_prob[in_bin])
            ece += np.abs(accuracy_in_bin - avg_confidence_in_bin) * prop_in_bin

    return ece

def benchmark_calibration(df: pd.DataFrame, model_name: str = 'XGBoost', drop_leakage: bool
= True, random_state: int = 42):
    """
    Evaluates probability calibration on out-of-fold predictions using 5-fold CV.
    Compares:
    - Uncalibrated Base Model
    - Platt Scaling (Sigmoid)
    - Isotonic Regression
    """
    X = df.drop(columns=['status'], errors='ignore')
    y = df['status'].values

    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=random_state)

    methods = ['Uncalibrated', 'Platt Scaling (Sigmoid)', 'Isotonic Regression']
    results = []

    for method in methods:
        fold_brier, fold_logloss, fold_ece, fold_auc = [], [], [], []

        for train_idx, val_idx in skf.split(X, y):
            X_train_raw, X_val_raw = df.iloc[train_idx], df.iloc[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            preprocessor, _, _ = create_preprocessing_pipeline(X_train_raw,
drop_leakage=drop_leakage, apply_igr=True)
            X_train_proc = preprocessor.fit_transform(X_train_raw)
            X_val_proc = preprocessor.transform(X_val_raw)

            base_model = get_candidate_models(random_state=random_state)[model_name]

            if method == 'Uncalibrated':
                cal_model = base_model
                cal_model.fit(X_train_proc, y_train)
            elif method == 'Platt Scaling (Sigmoid)':
                cal_model = CalibratedClassifierCV(estimator=base_model, method='sigmoid',
cv=3)
                cal_model.fit(X_train_proc, y_train)
            elif method == 'Isotonic Regression':
                cal_model = CalibratedClassifierCV(estimator=base_model, method='isotonic',
cv=3)
                cal_model.fit(X_train_proc, y_train)

```

Listing note: the final file contains additional execution and saving logic. The excerpt above is included to show the structure and the part most relevant to the report methodology.

H.5 src/fairness.py

```

"""
fairness.py - EquiLoan Research Framework
Phase 8 / Step 8: Demographic Bias Auditing & Fairness Analysis.
Evaluates Statistical Parity Difference (SPD) and Disparate Impact Ratio (DIR)

```

```

across sensitive / proxy demographic groups (`gender`, `region`, `age`).
"""

import os
import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedKFold
from data_validation import DataValidator
from preprocessing import create_preprocessing_pipeline
from baseline_models import get_candidate_models

def audit_demographic_fairness(df: pd.DataFrame, model_name: str = 'XGBoost',
sensitive_cols: list = ['gender', 'region', 'age'], drop_leakage: bool = True, random_state:
int = 42):
    """
    Audits model predictions for demographic disparities across sensitive columns.
    Target status: 0 = Approved, 1 = Rejected.
    Fitted via Stratified 5-Fold Cross-Validation.
    """
    X = df.drop(columns=['status'], errors='ignore')
    y = df['status'].values

    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=random_state)

    # Store predictions along with sensitive attributes
    df_results = df.copy()
    df_results['predicted_status'] = np.nan
    df_results['predicted_prob_approved'] = np.nan

    for train_idx, val_idx in skf.split(X, y):
        X_train_raw, X_val_raw = df.iloc[train_idx], df.iloc[val_idx]
        y_train, y_val = y[train_idx], y[val_idx]

        preprocessor, _, _ = create_preprocessing_pipeline(X_train_raw,
drop_leakage=drop_leakage, apply_iqr=True)
        X_train_proc = preprocessor.fit_transform(X_train_raw)
        X_val_proc = preprocessor.transform(X_val_raw)

        model = get_candidate_models(random_state=random_state)[model_name]
        model.fit(X_train_proc, y_train)

        preds = model.predict(X_val_proc)
        probs = model.predict_proba(X_val_proc)[:, 0] # probability of status 0 (Approved)

        df_results.iloc[val_idx, df_results.columns.get_loc('predicted_status')] = preds
        df_results.iloc[val_idx, df_results.columns.get_loc('predicted_prob_approved')] =
probs

    fairness_metrics = []

    for col in sensitive_cols:
        if col not in df_results.columns:
            continue

        group_rates = df_results.groupby(col)['predicted_status'].apply(lambda s: (s ==
0).mean())

        if len(group_rates) >= 2:
            max_rate = group_rates.max()
            min_rate = group_rates.min()
            max_group = group_rates.idxmax()
            min_group = group_rates.idxmin()

            spd = round(max_rate - min_rate, 4)
            dir_ratio = round(min_rate / max_rate, 4) if max_rate > 0 else np.nan

            adverse_impact_flag = "ALERT (DIR < 0.8)" if (not np.isnan(dir_ratio) and
dir_ratio < 0.8) else "PASS (DIR >= 0.8)"

            fairness_metrics.append({
                'Demographic Attribute': col,

```

```

        'Privileged Group (Max Rate)': f"{max_group} ({max_rate:.1%})",
        'Unprivileged Group (Min Rate)': f"{min_group} ({min_rate:.1%})",
        'Statistical Parity Diff (SPD)': spd,
        'Disparate Impact Ratio (DIR)': dir_ratio,
        'Adverse Impact Screening': adverse_impact_flag
    })

```

Listing note: the final file contains additional execution and saving logic. The excerpt above is included to show the structure and the part most relevant to the report methodology.

H.6 src/abstention.py

```

"""
abstention.py - EquiLoan Research Framework
Phase 10 / Step 11: Tri-State Decision-Confidence / Abstention Layer.
Instead of forcing binary decisions, classifies applicants as:
- High-confidence Approval (e.g. prob_deny < lower_thresh) -> APPROVE
- High-confidence Rejection (e.g. prob_deny > upper_thresh) -> REJECT
- Borderline / High Uncertainty -> HUMAN REVIEW
Analyzes Coverage, Accuracy at Coverage, Error Rate, and Review Percentage across threshold
grids.
"""

import os
import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.calibration import CalibratedClassifierCV
from data_validation import DataValidator
from preprocessing import create_preprocessing_pipeline
from baseline_models import get_candidate_models

def evaluate_abstention_layer(df: pd.DataFrame, model_name: str = 'XGBoost',
lower_thresholds: list = [0.20, 0.30, 0.35, 0.40], upper_thresholds: list = [0.80, 0.70,
0.65, 0.60], drop_leakage: bool = True, random_state: int = 42):
    """
    Evaluates tri-state decisions on out-of-fold predictions.
    Computes Coverage, Accuracy@Coverage, Error Rate@Coverage, and Review %.
    """
    X = df.drop(columns=['status'], errors='ignore')
    y = df['status'].values

    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=random_state)

    # Generate calibrated out-of-fold probabilities
    oof_probs = np.zeros(len(df))

    for train_idx, val_idx in skf.split(X, y):
        X_train_raw, X_val_raw = df.iloc[train_idx], df.iloc[val_idx]
        y_train, y_val = y[train_idx], y[val_idx]

        preprocessor, _, _ = create_preprocessing_pipeline(X_train_raw,
drop_leakage=drop_leakage, apply_iqr=True)
        X_train_proc = preprocessor.fit_transform(X_train_raw)
        X_val_proc = preprocessor.transform(X_val_raw)

        base_model = get_candidate_models(random_state=random_state)[model_name]
        cal_model = CalibratedClassifierCV(estimator=base_model, method='sigmoid', cv=3)
        cal_model.fit(X_train_proc, y_train)

        oof_probs[val_idx] = cal_model.predict_proba(X_val_proc)[: , 1] # P(status = 1 /
Denied)

    results = []

    # Baseline 100% Coverage (Forced Binary at 0.5 threshold)
    forced_preds = (oof_probs >= 0.5).astype(int)
    forced_acc = np.mean(forced_preds == y)
    forced_err = 1.0 - forced_acc

    results.append({
        'Threshold Config': 'Forced Binary (0.50 cutoff)',

```

```

'Lower Thresh (Approve)': 0.50,
'Upper Thresh (Reject)': 0.50,
'Automated Coverage %': 100.0,
'Human Review %': 0.0,
'Accuracy @ Coverage': round(forced_acc, 4),
'Error Rate @ Coverage': round(forced_err, 4),
'False Approval Rate': round(np.mean((forced_preds == 0) & (y == 1)), 4),
'False Rejection Rate': round(np.mean((forced_preds == 1) & (y == 0)), 4)
})

for low, high in zip(lower_thresholds, upper_thresholds):
    # Tri-state decision logic
    decisions = []
    for p in oof_probs:
        if p <= low:
            decisions.append('APPROVE')
        elif p >= high:
            decisions.append('REJECT')
        else:
            decisions.append('HUMAN REVIEW')

```

Listing note: the final file contains additional execution and saving logic. The excerpt above is included to show the structure and the part most relevant to the report methodology.

H.7 src/robustness.py

```

"""
robustness.py - EquiLoan Research Framework
Phase 14 / Step 16: Feature Stability & Robustness Analysis across Folds.
Computes cross-fold rank correlation (Spearman rho) and Top-10 feature overlap Jaccard index
to verify feature importance stability across Stratified K-Fold CV.
"""

import os
import pandas as pd
import numpy as np
from scipy.stats import spearmanr
from sklearn.model_selection import StratifiedKFold

from data_validation import DataValidator
from preprocessing import create_preprocessing_pipeline
from baseline_models import get_candidate_models

def calculate_feature_stability(df: pd.DataFrame, model_name: str = 'XGBoost', n_splits: int
= 5, drop_leakage: bool = True, random_state: int = 42):
    """
    Computes feature importance rankings across K folds and measures stability via:
    1. Spearman Rank Correlation (mean pairwise correlation across folds)
    2. Top-10 Jaccard Overlap Index
    """
    X = df.drop(columns=['status'], errors='ignore')
    y = df['status'].values

    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=random_state)

    fold_importances = []
    feature_names = None

    for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
        X_train_raw = df.iloc[train_idx]
        y_train = y[train_idx]

        preprocessor, num_cols, cat_cols = create_preprocessing_pipeline(X_train_raw,
drop_leakage=drop_leakage, apply_iqr=True)
        X_train_proc = preprocessor.fit_transform(X_train_raw)

        # Get feature names after one-hot encoding
        if feature_names is None:
            cat_encoder = preprocessor.named_transformers_['cat'].named_steps['encoder']
            cat_feature_names = cat_encoder.get_feature_names_out(cat_cols).tolist()
            feature_names = num_cols + cat_feature_names

```

```

model = get_candidate_models(random_state=random_state)[model_name]
model.fit(X_train_proc, y_train)

if hasattr(model, 'feature_importances_'):
    importances = model.feature_importances_
elif hasattr(model, 'coef_'):
    importances = np.abs(model.coef_[0])
else:
    continue

fold_importances.append(pd.Series(importances, index=feature_names))

df_imp = pd.DataFrame(fold_importances)

# 1. Spearman Rank Correlation matrix across folds
spearman_corrs = []
top10_jaccards = []

n_folds = len(fold_importances)
for i in range(n_folds):
    for j in range(i + 1, n_folds):
        # Spearman
        rho, _ = spearmanr(df_imp.iloc[i], df_imp.iloc[j])
        spearman_corrs.append(rho)

        # Top-10 Jaccard
        top_i = set(df_imp.iloc[i].nlargest(10).index)
        top_j = set(df_imp.iloc[j].nlargest(10).index)
        jaccard = len(top_i.intersection(top_j)) / len(top_i.union(top_j))
        top10_jaccards.append(jaccard)

```

Listing note: the final file contains additional execution and saving logic. The excerpt above is included to show the structure and the part most relevant to the report methodology.

H.8 src/evaluation.py

```

"""
evaluation.py - EquiLoan Research Framework
Phase 16 & 18 / Steps 20-21: Final Multi-Criteria Research Scorecard & Proposed vs. Baseline
Comparison.
Evaluates:
- Predictive Performance (ROC-AUC / F1) [30%]
- Calibration (Brier Score / ECE) [15%]
- Fairness (Disparate Impact Ratio DIR) [20%]
- Robustness / Stability (Spearman Rank Correlation) [15%]
- Interpretability & Counterfactual Recourse [10%]
- Computational Efficiency [10%]
Includes Sensitivity Analysis across alternative weighting schemes.
"""

import os
import pandas as pd
import numpy as np

def compute_research_scorecard(df_results: pd.DataFrame, weights: dict = None):
    """
    Computes a weighted Multi-Criteria Decision Analysis (MCDA) Research Scorecard.
    Normalized scores between 0 and 100 for each dimension.
    """
    if weights is None:
        weights = {
            'Performance': 0.30,
            'Calibration': 0.15,
            'Fairness': 0.20,
            'Robustness': 0.15,
            'Interpretability': 0.10,
            'Efficiency': 0.10
        }

    scored_rows = []

```

```

for idx, row in df_results.iterrows():
    # Score dimensions (0 - 100)
    perf_score = row['ROC_AUC'] * 100
    cal_score = max(0, (1.0 - row['Brier_Score']) * 100)
    fair_score = min(100, (row['DIR'] / 1.0) * 100) if not np.isnan(row['DIR']) else 50
    rob_score = row['Spearman_Stability'] * 100
    interp_score = row['Interp_Score'] # 0 - 100 rated
    eff_score = row['Efficiency_Score'] # 0 - 100 rated

    composite_score = (
        perf_score * weights['Performance'] +
        cal_score * weights['Calibration'] +
        fair_score * weights['Fairness'] +
        rob_score * weights['Robustness'] +
        interp_score * weights['Interpretability'] +
        eff_score * weights['Efficiency']
    )

    scored_rows.append({
        'Framework_Candidate': row['Framework_Candidate'],
        'ROC-AUC': round(row['ROC_AUC'], 4),
        'Brier Score': round(row['Brier_Score'], 4),
        'Disparate Impact Ratio (DIR)': round(row['DIR'], 4),
        'Feature Stability (Spearman)': round(row['Spearman_Stability'], 4),
        'Composite Research Score': round(composite_score, 2)
    })

return pd.DataFrame(scored_rows)

```

Listing note: the final file contains additional execution and saving logic. The excerpt above is included to show the structure and the part most relevant to the report methodology.

APPENDIX I: DETAILED LITERATURE NOTES

I.1 Haque and Hassan

This paper is the closest reference to our dataset because it reports 148,670 records and 37 attributes. The authors test Decision Tree, AdaBoost, Random Forest, SVM and Gaussian Naive Bayes, with AdaBoost reported at 99.99 percent accuracy. During the early stage of our project this result was treated as a target for performance. After we inspected the rate-related variables, the paper became more useful as a reason to check leakage than as a score to beat. Our final report therefore does not attempt to reproduce the near-perfect value. Instead, it reports the clean cross-validation benchmark after post-underwriting fields are excluded.

I.2 Lakshmi and Rao

The 2024 IJMTST paper presents a modernized loan approval system and uses ensemble learning ideas with Naive Bayes, Decision Tree and MLP. The paper describes the practical benefit of reducing the manual work involved in reviewing applications. It represents the type of end-to-end loan approval project that is common in student and applied work. EquiLoan keeps the same practical classification problem but adds explicit leakage, calibration, fairness and abstention experiments.

I.3 Sinap

The 2024 Gazi University study compares feature-selection techniques and reports that Recursive Feature Elimination improves model performance. Random Forest is the strongest model in that study, and cross-validation is described as more consistent than a single training, validation and testing division. This supports two choices in EquiLoan: keeping feature selection and stability as separate analytical questions and using stratified cross-validation for baseline model comparison.

I.4 Abdelhady et al.

This 2025 study uses 5,000 loan applicants and 14 features. Decision Tree, KNN and SVM are compared, with SVM reported at 96 percent accuracy. The result shows that model ranking depends on the dataset and feature representation. A method that is strongest on one public loan dataset may not be strongest on another. In our project, this is why the candidate set contains linear, tree, ensemble and probabilistic models rather than selecting XGBoost in advance.

I.5 Viswanatha et al.

This 2023 work compares Random Forest, Naive Bayes, Decision Tree and KNN. Naive Bayes is reported as the best model in their experiment with 83.73 percent accuracy. The study is useful because it reminds us that a simple model can be a meaningful baseline. EquiLoan includes Gaussian Naive Bayes even though tree ensembles are expected to perform well on mixed tabular features.

I.6 Bhavani

Bhavani presents loan approval as a conventional binary classification problem and discusses Logistic Regression, Random Forest and Naive Bayes. The paper is representative of the basic pipeline of collecting features, training models and returning a yes/no result. EquiLoan uses this type of system as the baseline from which the reliability-oriented additions are motivated.

I.7 Iqbal et al.

The 2024 study compares several ensemble methods and reports XGBoost accuracy of 0.9974 and AUC of 0.9998 on their dataset. It also studies feature importance. These results show that XGBoost and ensemble learning are already established in loan approval research. Our project therefore does not claim that XGBoost is a novel algorithm or that a high AUC by itself is a research contribution.

I.8 Akter et al.

The 2026 International Journal of Intelligent Systems paper is important for the novelty audit because it combines SHAP-guided feature selection, LIME interpretation, multiple classifiers and a web application. The authors report different performance across two datasets, including 86.17 percent accuracy with 81 percent AUC on one dataset and 99.06 percent accuracy on another. This work confirms that explainability and deployment are no longer sufficient novelty claims by themselves.

I.9 Sevimgin and Demirkol

The 2026 JAIDA paper evaluates nine algorithms, applies Grey Wolf Optimizer and develops an RF-LightGBM hybrid model with SHAP explanation. Their holdout accuracy is reported at 92.17 percent. The paper demonstrates that hybrid ensembles, metaheuristic optimization and explainability can already be combined in a modern loan prediction pipeline. EquiLoan therefore keeps its contribution focused on how leakage, balancing, calibration, fairness, abstention and stability are evaluated together.

I.10 Overall Literature Position

Across the referred papers, the common theme is improving predictive accuracy and, in newer work, adding feature selection or explainability. The literature does not justify a claim that our selected algorithms or explainability tools are new. The defensible student-project novelty is the experiment design. We audit feature timing, compare two class-distribution conditions, test probability calibration, report group-level differences, introduce a human-review band and preserve version history. This is an integration and evaluation contribution rather than a new mathematical model.

APPENDIX J: NOTEBOOK VERSION HISTORY AND PROJECT EVOLUTION

Notebook file	Total cells	Markdown	Code	Outputs
Copy_of_EquiLoan_Master_Notebook_(2).ipynb	50	12	38	48
EquiLoan_Fixed_Final (3).ipynb	64	22	42	55
EquiLoan_Fixed_Final.ipynb	64	22	42	0
EquiLoan_Master_Notebook_FINAL_(1) (2).ipynb	65	13	52	34
EquiLoan_Master_Notebook_FINAL_(1).ipynb	61	12	49	61

The version inventory shows that the project was not produced in one final notebook. Different copies contain different levels of explanation, outputs and modeling choices. Keeping the versions is useful because early figures show the exact problems that later motivated leakage pruning and modular source files.

J.1 Development Decision Record

Initial notebook stage: inspect the dataset, visualize target balance and missing values, encode the data and compare several classifiers.

High-score stage: early model outputs produced very strong ROC curves and performance values. At this stage the project still treated the dataset largely as a normal classification problem.

Leakage-review stage: missingness was compared by target status. Rate-related fields were found to be strongly associated with the underwriting outcome and were moved to the leakage list.

Modularisation stage: dataset validation, preprocessing, baseline evaluation, calibration, fairness, abstention, robustness and scorecard logic were separated into Python files.

Final-report stage: all main figures and tables were regenerated from the saved clean result CSV files. Early notebook graphs were retained only as development evidence.

J.2 Why the Final Result Differs from Early Notebook Graphs

Several early graphs show accuracy and ROC-AUC values that are much higher than the final clean benchmark. This difference is intentional. Once the project identified rate-related fields as post-underwriting leakage, those variables were excluded and the models were re-evaluated. The final report therefore uses the lower leakage-pruned values. This is a stronger scientific position because it explains why a result changed instead of hiding the earlier version.

APPENDIX K: METRIC DEFINITIONS USED IN THE PROJECT

K.1 Accuracy

Accuracy is the number of correct predictions divided by the total number of predictions. It is easy to understand but can be misleading when one class is much larger than the other. For the natural EquiLoan dataset, about three quarters of the records are approved, so accuracy must be read together with recall and F1.

K.2 Precision

Precision for status 1 is TP divided by TP plus FP. It answers how many of the applications predicted as rejected are actually rejected in the historical labels.

K.3 Recall

Recall is TP divided by TP plus FN. It measures how many of the historical rejected applications are detected by the model. The balanced experiment increases recall for XGBoost compared with the natural condition.

K.4 F1 Score

F1 is the harmonic mean of precision and recall. It is useful when both false positives and false negatives matter and when the class distribution is uneven.

K.5 ROC-AUC

ROC-AUC measures ranking across decision thresholds. A higher value means the model tends to give higher scores to positive records than negative records. It does not by itself show whether the probability values are calibrated.

K.6 Brier Score

Brier score is the mean squared difference between predicted probability and the observed binary outcome. Lower values indicate better probability quality. It is used in both the baseline and calibration experiments.

K.7 Log Loss

Log loss places a larger penalty on highly confident wrong probabilities. It is reported in the calibration module to complement the Brier score.

K.8 Expected Calibration Error

ECE divides predictions into confidence bins and compares average confidence with observed outcome frequency. Lower values indicate smaller average calibration gaps, but the value depends on the binning procedure.

K.9 Statistical Parity Difference

SPD is the difference between the highest and lowest observed positive prediction rates in the group audit used by this project. A larger difference indicates a larger gap in model output rates.

K.10 Disparate Impact Ratio

DIR is the lower positive prediction rate divided by the higher positive prediction rate. The project uses 0.80 as a screening line only. The metric does not prove discrimination and does not identify why a difference exists.

K.11 Automated Coverage

Coverage is the percentage of records for which the abstention system produces an automatic decision. The remaining records are sent to the review band.

K.12 Accuracy at Coverage

This is the accuracy measured only on the records that remain after uncertain cases are removed. It should always be reported together with coverage because a model can improve this accuracy by sending more cases to review.

K.13 Feature Stability

The robustness module stores mean importance, standard deviation and coefficient of variation across folds. A small coefficient of variation indicates that a feature has a more stable importance estimate within the tested model and data condition.